[University Name]

[Faculty / Department]

MASTER'S THESIS

Study on the Parameter Optimization of Object Tracking Algorithm Based on Intelligent Optimization Method

Submitted in partial fulfilment of the requirements for the degree of

Master of Science in Computer Science and Technology

Author: [Student Name]

Supervisor: [Supervisor Name]

2026

Abstract

Multi-object tracking (MOT) aims to locate every object of interest in a video and maintain a consistent identity for each object over time. Tracking-by-detection algorithms such as SORT, DeepSORT, ByteTrack and OC-SORT are the dominant approach because of their simplicity and speed. Their performance, however, depends strongly on a set of hand-tuned hyper-parameters: detection confidence thresholds, association IoU thresholds, Kalman filter noise covariances, track birth and death parameters, and the weights that balance motion and appearance cues. In practice these parameters are set by trial and error or copied from default values tuned on a different dataset, which leads to sub-optimal and unstable performance when the tracker is deployed in a new scene.

This thesis studies the automatic optimization of tracking parameters using intelligent (swarm and evolutionary) optimization methods. Parameter tuning is formulated as a black-box, mixed continuous–integer optimization problem whose objective is a tracking quality metric (HOTA, MOTA or IDF1) evaluated on a validation video set. Because each objective evaluation requires running the tracker over many video frames, the problem is expensive, noisy and non-differentiable, making gradient-based methods unsuitable.

The main contributions are as follows. (1) A systematic analysis of the parameter space of a representative tracker (ByteTrack-style association with a Kalman motion model and optional appearance features) is performed, identifying the parameters to which tracking quality is most sensitive. (2) An improved particle swarm optimization algorithm (IPSO) is proposed, which combines chaotic (logistic map) initialisation, nonlinear decreasing inertia weight, adaptive acceleration coefficients and a Lévy-flight mutation of the global best to escape local optima. (3) To reduce the cost of objective evaluation, a surrogate-assisted strategy and a caching mechanism that reuses detector outputs across evaluations are designed, so that only the association stage is re-run for each candidate parameter set. (4) A multi-objective extension is presented that simultaneously optimizes accuracy (HOTA) and identity stability (number of ID switches). Experiments are designed on the MOT17 and MOT20 benchmarks to compare IPSO with default parameters, grid search, random search, the genetic algorithm (GA), standard PSO, grey wolf optimization (GWO) and Bayesian optimization (TPE), and to evaluate generalisation of the tuned parameters to unseen sequences.

Keywords: multi-object tracking; parameter optimization; particle swarm optimization; genetic algorithm; swarm intelligence; Kalman filter; data association; HOTA.

Table of Contents

Abstract **2**

Chapter 1 Introduction **6**

1.1 Research Background 6

1.2 Research Significance 6

1.3 Research Status at Home and Abroad 7

1.4 Research Content 16

1.5 Thesis Organisation 16

Chapter 2 Theoretical Foundations **18**

2.1 Tracking-by-Detection Framework 18

2.2 Kalman Filter Motion Model 18

2.3 Data Association 19

2.4 Tracking Evaluation Metrics 20

2.5 Intelligent Optimization Methods 20

2.6 Chapter Summary 21

Chapter 3 Problem Formulation and Parameter Analysis **22**

3.1 Baseline Tracker 22

3.2 Parameter Space 22

3.3 Mechanical Role of Each Parameter 22

3.4 Optimization Model 25

3.5 Data Split for Fair Optimization 26

3.6 Parameter Sensitivity Analysis 26

3.7 Chapter Summary 27

Chapter 4 Improved PSO for Tracker Parameter Optimization **28**

4.1 Limitations of Standard PSO 28

4.2 Chaotic Initialisation 28

4.3 Nonlinear Adaptive Inertia Weight 28

4.4 Asynchronous Acceleration Coefficients 29

4.5 Lévy-Flight Mutation of the Global Best 29

4.6 Convergence and Stability Analysis of PSO 30

4.7 Handling Integer and Constrained Parameters 32

4.8 Efficient Objective Evaluation 32
4.9 Algorithm Procedure 35
4.10 Detailed IPSO Pseudocode 36
4.11 Complexity 37
4.12 Chapter Summary 38
Chapter 5 Multi-Objective Parameter Optimization 39
5.1 Motivation 39
5.2 Formulation 39
5.3 Multi-Objective IPSO 39
5.4 Decision Making 41
5.5 Chapter Summary 42
Chapter 6 Experiments and Analysis 43
6.1 Experimental Setup 43
6.2 Single-Objective Optimization Results 44
6.3 Statistical Testing Protocol 46
6.4 Convergence Analysis 48
6.5 Benchmark Function Validation 48
6.6 Ablation Study of IPSO 49
6.7 Generalisation Experiments 50
6.8 Multi-Objective Results 51
6.9 Discussion of Optimized Parameters 51
6.10 Threats to Validity 52
6.11 Chapter Summary 53
Chapter 7 Conclusion and Future Work 54
7.1 Conclusions 54
7.2 Limitations 54
7.3 Future Work 54
References 55
Acknowledgements 60

Chapter 1
Introduction

1.1 Research Background

Video-based object tracking is a fundamental task in computer vision with applications in intelligent surveillance, autonomous driving, sports analysis, robotics, crowd management and traffic monitoring. With the development of deep learning-based object detectors, the tracking-by-detection paradigm has become dominant: an object detector is applied to each frame, and a tracker associates detections across frames to form trajectories.

A typical tracking-by-detection algorithm involves several components — a motion model (usually a Kalman filter), an affinity measure (IoU, Mahalanobis distance, appearance embedding distance), an assignment algorithm (the Hungarian algorithm), and track management rules for creating, confirming and deleting tracks. Each component introduces parameters. For example, ByteTrack requires a high detection threshold, a low detection threshold, a new-track threshold, a matching threshold and a track buffer length; DeepSORT additionally requires the maximum cosine distance for appearance matching, the gating threshold and the weight between motion and appearance.

These parameters interact in complex ways. A lower detection threshold recovers occluded objects but introduces false positives; a longer track buffer allows re-identification after occlusion but increases the risk of incorrect re-association; a tighter matching threshold reduces identity switches but fragments trajectories. Moreover, optimal values depend on the scene: camera motion, frame rate, object density and detector quality all affect the best setting. Manual tuning is therefore laborious, requires expert knowledge, and is rarely optimal.

Intelligent optimization methods — evolutionary algorithms such as the genetic algorithm (GA) and differential evolution (DE), and swarm intelligence algorithms such as particle swarm optimization (PSO), ant colony optimization (ACO) and grey wolf optimization (GWO) — are population-based, derivative-free global search methods. They are well suited to black-box problems in which the objective can only be evaluated, not differentiated. This thesis applies and improves such methods for automatic parameter optimization of object trackers.

1.2 Research Significance

- **Practical significance:** automatic tuning reduces the engineering effort required to deploy a tracker in a new scene and improves tracking quality without changing the detector or the tracker architecture, i.e., at zero additional inference cost.
- **Scientific significance:** the study provides a quantitative understanding of the sensitivity of tracking quality to each parameter and of parameter interactions, and evaluates the suitability of different intelligent optimizers for expensive, noisy, mixed-variable problems.
- **Methodological significance:** the improved PSO and the evaluation-cost reduction strategies are general and can be applied to other computer vision pipelines with hand-tuned parameters.

1.3 Research Status at Home and Abroad

1.3.1 Classical multi-target tracking

The data association problem was studied in radar and sonar surveillance long before video tracking became practical. Reid proposed multiple hypothesis tracking (MHT) [31], which maintains a tree of alternative association hypotheses over several scans and defers hard decisions until later observations resolve the ambiguity. MHT is theoretically attractive because it can recover from early association errors, but the number of hypotheses grows exponentially with the number of targets and frames, so practical implementations depend on aggressive pruning, gating and N-scan truncation. Kim et al. revisited MHT for video [34] and showed that, when combined with an online-learned appearance model and careful pruning, it remained competitive on the MOT benchmarks of the time; their study also showed that the gating and pruning thresholds strongly influence accuracy, which is an early indication of the parameter-tuning problem addressed in this thesis.

Joint probabilistic data association (JPDA), introduced by Fortmann, Bar-Shalom and Scheffe [32], avoids explicit hypothesis enumeration by computing, for each target, the marginal probability that each measurement originated from it, and by updating the target state with a probability-weighted combination of measurements. JPDA is well suited to cluttered scenes with a fixed number of targets, but it does not natively handle track birth and death and its cost also grows combinatorially with the number of targets in a gate.

A third family formulates tracking as a global (batch) optimisation over the whole video. Zhang, Li and Nevatia cast data association as a min-cost network flow problem [33], in which detections are nodes, transitions between detections are edges with costs derived from affinity, and source and sink edges model track birth and death. The globally

optimal set of trajectories can then be found in polynomial time. Network-flow and related graph formulations (minimum-cost multicut, lifted multicut and conditional random field models) achieved strong offline results, but they require the whole sequence (or a long window) and are therefore unsuitable for online applications. All these classical methods contain cost weights, birth and death costs and gating thresholds that are typically set by hand or by small grid searches.

1.3.2 Online tracking-by-detection: the SORT family

The rapid improvement of CNN detectors shifted the bottleneck of tracking from association modelling to detection quality. Bewley et al. demonstrated with SORT [1] that a very simple online tracker — a constant-velocity Kalman filter [14], IoU affinity and Hungarian assignment [15] — achieved accuracy comparable to far more complex methods when given a strong detector, while running at hundreds of frames per second. SORT has only a handful of parameters (detection threshold, IoU threshold, maximum age and minimum hits), but its accuracy was shown to be very sensitive to the detection threshold.

DeepSORT [2] added a deep appearance descriptor trained on a person re-identification dataset, a Mahalanobis gating step based on the Kalman covariance, and a matching cascade that gives priority to more recently seen tracks. These additions considerably reduced identity switches during occlusion but introduced further parameters: the maximum cosine distance, the gating threshold, the appearance gallery size (budget) and the cascade depth (maximum age).

ByteTrack [3] observed that discarding low-confidence detections throws away many occluded but true objects. It therefore associates high-score detections first and then matches the remaining tracks with low-score detections using IoU only, which recovers occluded objects while the motion consistency check suppresses most background false positives. With a YOLOX detector [30], ByteTrack achieved state-of-the-art results on MOT17 and MOT20 [12, 13] without any appearance model. Its public implementation, however, uses different thresholds for different benchmarks and even for individual sequences, which illustrates that its parameters are scene-dependent.

OC-SORT [4] analysed the failure modes of the Kalman filter when an object is not observed: the linear motion prediction accumulates error during occlusion and the track drifts. OC-SORT introduces observation-centric re-update (re-running the filter along a virtual trajectory once the object is re-detected), observation-centric momentum (a direction-consistency term in the association cost) and observation-centric recovery. BoT-SORT [5] adds camera motion compensation by global motion estimation, replaces the Kalman state with box width and height instead of aspect ratio, and fuses IoU and

appearance distances. StrongSORT [6] revisits DeepSORT with a stronger detector and embedding, an exponential moving average of appearance features, an NSA Kalman filter whose measurement noise is scaled by detection confidence, and two lightweight post-processing modules (AFLink and Gaussian-smoothed interpolation). Deep OC-SORT [40] extends OC-SORT with camera motion compensation, dynamic appearance weighting according to detection confidence and an adaptive weighting of the appearance term. Hybrid-SORT [41] argues that weak cues — detection confidence state and height state — complement strong spatial and appearance cues and adds them to the association cost with little extra computation.

Each of these trackers adds components, and each component adds tunable parameters: BoT-SORT has proximity and appearance thresholds, Deep OC-SORT has an embedding-weighting temperature and an alpha-fixed EMA factor, Hybrid-SORT has weights for the confidence and height cues. The number of parameters has therefore grown steadily, while the procedure for setting them has remained largely manual.

1.3.3 Joint detection-and-tracking and transformer-based trackers

A second line of work integrates detection and tracking into a single network. Tracktor [35] uses the regression head of a Faster R-CNN detector to propagate boxes from frame to frame, effectively turning the detector into a tracker. CenterTrack [36] predicts object centres and their offsets to the previous frame, conditioned on the previous frame and its heatmap. JDE [37] adds an embedding head to a one-stage detector so that detection and appearance features are produced in one forward pass, and FairMOT [7] identifies that anchor-based detection biases the re-identification branch and uses an anchor-free CenterNet-style design with balanced detection and re-identification branches.

Transformer-based trackers model association with attention. TransTrack [39] uses object queries for detection and track queries carried from the previous frame. TrackFormer [8] and MOTR [38] propagate track queries over time so that each query follows one object, and new objects are detected by fresh detect queries; MOTR trains the whole process end-to-end with a collective average loss over video clips. These methods reduce the number of hand-designed association rules, but they still contain inference thresholds (query score thresholds for birth and death, the number of frames a query may be kept alive without a match) and they have been found to lag behind well-tuned tracking-by-detection methods on crowded benchmarks such as MOT20, while being considerably more expensive to train. DanceTrack [42], a benchmark with uniform appearance and diverse motion, was proposed precisely to stress association rather than detection, and on it the relative ranking of methods differs markedly from MOT17.

The development of MOT can therefore be summarised as follows: classical methods reasoned explicitly about association uncertainty; the SORT family demonstrated that simple online association with a strong detector is highly effective; joint and transformer trackers moved association into the network. Throughout this evolution, the tracking-by-detection family has remained the practical default, and its quality remains tied to a set of hand-set thresholds and weights.

1.3.4 Evolution of tracking evaluation metrics

Early tracking papers reported ad hoc measures such as the number of mostly tracked (MT) and mostly lost (ML) trajectories, fragmentations and identity switches. Bernardin and Stiefelhagen defined the CLEAR MOT metrics [9]: MOTA aggregates false negatives, false positives and identity switches into one accuracy number, and MOTP measures localisation precision of the matched pairs. MOTA was adopted as the main ranking metric of MOTChallenge [12], which strongly influenced tracker design: because the number of detections far exceeds the number of identity switches, MOTA is dominated by detection quality, and trackers were tuned to maximise recall.

Ristani et al. introduced the identity-based measures IDP, IDR and IDF1 [10], which compute a one-to-one bipartite matching between whole ground-truth and predicted trajectories and measure the fraction of correctly identified detections. IDF1 rewards long-term identity consistency but is less sensitive to detection quality, and its global matching can produce unintuitive results when a trajectory is split into several parts of similar length.

Luiten et al. proposed HOTA [11] to resolve the imbalance between these metrics. HOTA is the geometric mean of detection accuracy (DetA) and association accuracy (AssA), evaluated at a series of localisation thresholds and averaged, and it decomposes further into detection recall and precision, association recall and precision and localisation accuracy (LocA). HOTA has been adopted as the main metric by MOTChallenge, KITTI tracking and DanceTrack, and the TrackEval library provides a reference implementation of HOTA, CLEAR and identity metrics. For the purposes of this thesis, the choice of metric is not neutral: optimising MOTA, IDF1 or HOTA leads to different optimal parameters, which motivates both the use of HOTA as the primary single objective and the multi-objective extension in Chapter 5.

1.3.5 Evolutionary algorithms

The genetic algorithm (GA), formalised by Holland [18], encodes candidate solutions as chromosomes and evolves a population through fitness-proportional or tournament selection, crossover and mutation. For real-valued problems, simulated binary crossover and polynomial mutation are commonly used, and elitism guarantees that the best

solution is never lost. GA is robust and easy to parallelise, but its convergence on continuous problems is relatively slow and its performance depends on crossover and mutation rates.

Differential evolution (DE), proposed by Storn and Price [19], creates a mutant vector by adding a scaled difference of two population members to a third ($v = x_{r1} + F\,(x_{r2} - x_{r3})$), recombines it with the target vector by binomial crossover with rate CR, and keeps the better of the trial and target vectors. The difference-vector mechanism makes the step size adapt automatically to the spread of the population, which gives DE strong performance on continuous benchmarks. Self-adaptive variants such as jDE, JADE and SHADE adapt F and CR during the run.

Evolution strategies form a third branch. The covariance matrix adaptation evolution strategy (CMA-ES) of Hansen and Ostermeier [53] samples offspring from a multivariate normal distribution and adapts the mean, the global step size (via cumulative step-size adaptation) and the full covariance matrix (via rank-one and rank-μ updates) from the successful steps. CMA-ES is invariant to rotations and order-preserving transformations of the objective, learns variable correlations, and is often regarded as the state of the art for continuous black-box optimisation in moderate dimensions. Its default population size, $4 + \lfloor 3 \ln n \rfloor$, is small, which suits expensive problems, but its behaviour on integer variables and on plateau-rich landscapes requires additional handling such as integer step-size lower bounds.

1.3.6 Particle swarm optimisation and its variants

Kennedy and Eberhart introduced PSO [16] as a simulation of social behaviour in bird flocks. Shi and Eberhart added the inertia weight ω [17] and recommended a linear decrease from 0.9 to 0.4. Clerc and Kennedy analysed the swarm as a dynamic system [44] and derived the constriction factor $\chi = 2 / | 2 - \varphi - \mathrm{sqrt}(\varphi^2 - 4\varphi) |$ with $\varphi = c_1 + c_2 > 4$; with the common choice $\varphi = 4.1$, $\chi \approx 0.7298$, which is equivalent to an inertia weight of 0.7298 and acceleration coefficients of about 1.496. Trelea [45] studied the deterministic dynamics of a single particle and mapped the regions of the (ω, c) plane that lead to convergent, oscillatory and zigzagging behaviour. Ratnaweera et al. [46] proposed time-varying acceleration coefficients (TVAC), with c_1 decreasing from 2.5 to 0.5 and c_2 increasing from 0.5 to 2.5, which is the schedule adopted by the IPSO in this thesis.

Several variants address premature convergence. Comprehensive learning PSO (CLPSO) by Liang et al. [47] lets each dimension of a particle learn from the personal best of a different, tournament-selected particle, which preserves diversity and greatly improves performance on multimodal functions at the cost of slower convergence on unimodal

ones. Chaotic PSO variants replace uniform random numbers or the initial population with sequences from chaotic maps such as the logistic or tent map; Alatas et al. [68] systematically compared several chaotic maps embedded in PSO parameters and reported improvements in solution quality and convergence. Quantum-behaved PSO (QPSO) by Sun et al. [48] removes the velocity term and samples each particle's new position around a local attractor from a distribution derived from a delta potential well, controlled by a single contraction–expansion coefficient; QPSO has fewer parameters and stronger global search ability in many studies. Topological variants (ring, von Neumann neighbourhoods) slow information flow to delay convergence, and hybrid variants combine PSO with DE or local search.

1.3.7 Other swarm intelligence algorithms

Ant colony optimisation (ACO) by Dorigo et al. [49] constructs solutions incrementally using pheromone trails that are reinforced by good solutions and evaporate over time. ACO is naturally suited to combinatorial problems such as routing; continuous extensions such as ACO_R sample from Gaussian kernels centred on archived solutions. The artificial bee colony (ABC) of Karaboga and Basturk [50] divides the population into employed, onlooker and scout bees: employed and onlooker bees perform neighbourhood search around food sources (with onlookers selecting sources proportionally to fitness), and scouts replace sources that have not improved within a limit, which provides an explicit restart mechanism.

Mirjalili and co-workers proposed a series of nature-inspired algorithms. The grey wolf optimiser (GWO) [20] moves each wolf towards the average of positions dictated by the three best wolves α, β and δ, with a coefficient a decreasing from 2 to 0 controlling the transition from exploration to exploitation. The whale optimisation algorithm (WOA) [51] alternates between encircling the best solution, a logarithmic spiral update that imitates bubble-net feeding, and a random search towards a randomly chosen whale. The salp swarm algorithm (SSA) [52] arranges the population in a chain: the leader moves around the food source (best solution) and followers update their position as the average of their own and the preceding salp's position. These algorithms are simple and widely applied, but several critical studies have noted that many recent "metaphor-based" algorithms share mechanisms with PSO or DE, and that some perform well on benchmarks partly because of a bias towards the origin of the search space. This thesis therefore treats GWO as a representative modern swarm method and compares against it under identical budgets, rather than adopting a new metaphor.

1.3.8 Hyper-parameter optimisation

Grid search evaluates the Cartesian product of a few values per parameter. It is simple and reproducible, but its cost grows exponentially with dimension, and when only a few parameters matter most grid points duplicate the same values of the important parameters. Bergstra and Bengio showed that random search [24] explores each individual dimension more densely for the same budget and is therefore more efficient when the effective dimensionality is low.

Bayesian optimisation (BO) fits a probabilistic surrogate to all observations and selects the next point by maximising an acquisition function that balances predicted quality and uncertainty. Jones et al. introduced the efficient global optimisation (EGO) algorithm [54] using a Gaussian process (kriging) surrogate [64] and the expected improvement criterion. Snoek et al. [55] popularised GP-based BO for machine learning hyper-parameters, introducing integrated acquisition over GP hyper-parameters and cost-aware and parallel variants. Bergstra et al. [25] proposed the tree-structured Parzen estimator (TPE), which models the densities of good and bad configurations, $l(\theta)$ and $g(\theta)$, separately and maximises their ratio; TPE handles conditional and discrete parameters naturally and underlies Hyperopt and the default sampler of Optuna [26]. SMAC by Hutter et al. [56] uses a random forest [63] as the surrogate, which scales better to many observations and handles categorical parameters, and was designed for general algorithm configuration, where the objective is the performance of an algorithm over a set of problem instances — a setting closely analogous to tracker tuning over a set of video sequences.

Multi-fidelity methods reduce cost by evaluating many configurations cheaply and promoting only the promising ones. Successive halving [57] evaluates n configurations with a small budget, keeps the best $1/\eta$ fraction, multiplies the budget by η, and repeats. Hyperband [58] runs successive halving with several trade-offs between the number of configurations and the minimum budget, hedging against the risk that low-budget evaluations are misleading. BOHB [59] replaces Hyperband's random sampling with a TPE-like model, combining the fast early progress of Hyperband with the strong final performance of BO. For tracking, a natural fidelity is the number of frames or sequences processed, although the correlation between short-clip and full-sequence metrics must be verified, since association errors often arise only in long occlusions.

1.3.9 Surrogate-assisted evolutionary algorithms

Surrogate-assisted evolutionary algorithms (SAEAs) embed an approximate model into an evolutionary or swarm search to reduce the number of expensive evaluations. Jin's survey [60] distinguishes global surrogates, which approximate the whole landscape, from local surrogates built around the current population, and identifies model

management — deciding which individuals are evaluated with the true objective — as the central issue. Common strategies include pre-selection (generate many offspring, evaluate only those ranked best by the surrogate), individual-based control (evaluate a fixed fraction), generation-based control (alternate surrogate and true generations) and infill criteria borrowed from BO such as expected improvement or lower confidence bound. Popular surrogates are polynomial response surfaces, radial basis functions, Gaussian processes and random forests. The pre-screening strategy of IPSO in Chapter 4 follows the pre-selection approach, which keeps the swarm dynamics intact and uses the surrogate only to filter candidates.

1.3.10 Multi-objective evolutionary algorithms

NSGA-II by Deb et al. [21] ranks the population by fast non-dominated sorting, breaks ties by crowding distance, and uses elitist ($\mu + \lambda$) selection; it remains the most widely used multi-objective evolutionary algorithm (MOEA) for two and three objectives. NSGA-III [61] replaces crowding distance by a set of well-spread reference points on a normalised hyperplane, which maintains diversity for many objectives. MOEA/D by Zhang and Li [62] decomposes the multi-objective problem into scalar sub-problems (weighted sum, Tchebycheff or boundary intersection) with neighbouring weight vectors, and optimises them simultaneously with information exchange between neighbours. MOPSO by Coello Coello et al. [22] extends PSO with an external archive of non-dominated solutions, an adaptive grid for diversity, and a mutation operator; leaders are selected from less crowded regions of the archive. Performance of MOEAs is usually measured by the hypervolume indicator introduced by Zitzler and Thiele [67], which is the only widely used unary indicator that is strictly Pareto-compliant, together with spread and convergence measures.

1.3.11 Hyper-parameter optimisation for trackers

Systematic studies of tracker parameter optimisation remain limited. Most tracker papers report a single set of hand-selected values, sometimes with per-sequence adjustments; ByteTrack, for example, adjusts the track threshold for specific MOT17 and MOT20 sequences in its released code. Some works perform small grid searches over one or two thresholds in ablation studies, and practitioners increasingly use generic HPO libraries such as Optuna to tune trackers, but these efforts are rarely reported with the rigour expected of optimisation studies (multiple seeds, equal budgets, statistical testing, held-out validation). The YOLOv5 project evolves training hyper-parameters with a simple mutation-based GA, which shows that evolutionary tuning is accepted in practice for vision pipelines, but its target is detector training rather than association.

1.3.12 Comparison of optimisation approaches

Method	Key mechanism	Strengths	Weaknesses for tracker tuning
Grid search	Exhaustive lattice	Simple, reproducible	Exponential cost; wastes budget on unimportant parameters
Random search [24]	Uniform sampling	Embarrassingly parallel; robust	No exploitation of good regions
GA [18]	Selection, crossover, mutation	Robust, handles mixed variables	Slow convergence on continuous problems
DE [19]	Difference-vector mutation	Self-scaling steps; strong on continuous problems	Many evaluations; sensitive to F and CR
CMA-ES [53]	Adaptive Gaussian sampling	Learns correlations; invariant	Weak on integer and plateau landscapes
PSO [16, 17]	Velocity and social learning	Few parameters; fast convergence	Premature convergence
CLPSO / QPSO [47, 48]	Diversity-preserving learning	Better on multimodal problems	Slower convergence; more evaluations
GWO / WOA / SSA [20, 51, 52]	Leader-guided encircling	Simple, few parameters	Bias towards centre; limited theory
ACO / ABC [49, 50]	Pheromone / bee foraging	Restarts (ABC); combinatorial (ACO)	Continuous versions less efficient
GP-BO / TPE [54, 25]	Surrogate and acquisition	Sample-efficient	Sequential; scales poorly with batch size
SMAC [56]	Random forest surrogate	Handles instances and categories	Weak uncertainty estimates
Hyperband / BOHB [58, 59]	Multi-fidelity early stopping	Very cheap low-fidelity screening	Requires a reliable low-fidelity proxy
NSGA-II / MOEA/D / MOPSO [21, 62, 22]	Pareto-based search	Trade-off fronts	Larger budgets required

Table 1.1 Comparison of optimisation approaches considered for tracker parameter tuning.

1.3.13 Summary of existing problems and research gap

The review above reveals the following problems and gaps, which the present thesis addresses.

- **Manual and non-transferable parameters.** Trackers from SORT to Hybrid-SORT add tunable components, but their parameters are set manually, often per benchmark

or per sequence, and the transfer of parameters between scenes is rarely evaluated.

- **Expensive evaluation budget.** Each objective evaluation runs the tracker over thousands of frames. Generic HPO studies in deep learning assume either cheap evaluations (benchmarks) or multi-fidelity proxies (training epochs) that do not directly exist for association.
- **Premature convergence of standard swarms.** The tracking objective is plateau-rich and multimodal; standard PSO with small populations collapses early, while diversity-preserving variants (CLPSO) are too slow for small budgets. A variant that is efficient at small budgets but able to escape plateaus is required.
- **Noisy and piecewise-constant objectives.** Threshold-based decisions make the objective piecewise constant, and small changes cause spurious fluctuations; most swarm algorithms assume a deterministic, smooth objective and greedily accept tiny improvements.
- **Single-metric bias.** Optimising MOTA favours detection, optimising IDF1 favours association; existing tuning practice ignores this trade-off, which multi-objective methods could expose.
- **Lack of rigorous comparison.** There is no study that compares grid search, random search, evolutionary, swarm and Bayesian methods for tracker tuning under equal budgets with statistical testing and generalisation analysis.

Accordingly, this thesis combines a lightweight improved PSO with surrogate pre-screening, noise-robust selection and detection caching, and evaluates it against representative methods from each family under a common protocol.

1.4 Research Content

- Formulate tracker parameter tuning as an expensive black-box optimization problem and analyse parameter sensitivity.
- Propose an improved PSO algorithm with chaotic initialisation, adaptive coefficients and Lévy-flight mutation.
- Design an efficient evaluation framework using cached detections, parallel evaluation and surrogate pre-screening.
- Extend the method to multi-objective optimization of accuracy and identity stability.
- Design and conduct comparison, convergence, generalisation and ablation experiments.

1.5 Thesis Organisation

Chapter 2 introduces tracking and optimization theory. Chapter 3 formulates the parameter optimization problem and analyses sensitivity. Chapter 4 presents the improved PSO algorithm and the efficient evaluation framework. Chapter 5 presents the multi-objective extension. Chapter 6 describes the experiments. Chapter 7 concludes.

Chapter 2
Theoretical Foundations

2.1 Tracking-by-Detection Framework

At each frame t, a detector produces a set of detections $D_t = \{d_1, ..., d_n\}$, each with a box and confidence score. The tracker maintains a set of tracks T_{t-1}. Each frame is processed in four steps: (1) **prediction** of each track's state with the motion model; (2) **affinity computation** between predicted tracks and detections; (3) **assignment** by solving a linear assignment problem; (4) **track management** — updating matched tracks, creating new tracks from unmatched detections and deleting tracks that have been unmatched for too long.

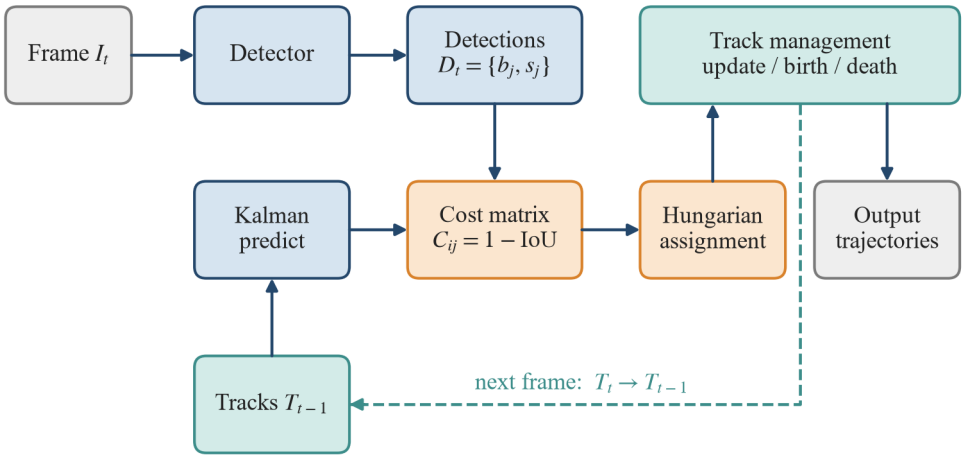


Figure 2.1 Tracking-by-detection loop: per-frame detection, Kalman prediction of existing tracks, cost-matrix construction, Hungarian assignment and track management.

2.2 Kalman Filter Motion Model

The state of a track is commonly defined as $x = [u, v, s, r, u', v', s']^T$ (SORT) or $[x, y, a, h, x', y', a', h']^T$ (DeepSORT/ByteTrack), where (u, v) is the box centre, s the area, r or a the aspect ratio and h the height. The constant-velocity model is:

$$x_k = F\,x_{k-1} + w_k,\; w_k \sim N(0, Q) \tag{2.1}$$

$$z_k = H\,x_k + v_k,\; v_k \sim N(0, R) \tag{2.2}$$

Prediction and update follow the standard equations:

$$\hat{x}_{k|k-1} = F\,\hat{x}_{k-1},\; P_{k|k-1} = F\,P_{k-1}\,F^T + Q \tag{2.3}$$

$$K_k = P_{k|k-1}\,H^T\,(H\,P_{k|k-1}\,H^T + R)^{-1} \tag{2.4}$$

$$\hat{x}_k = \hat{x}_{k|k-1} + K_k\,(z_k - H\,\hat{x}_{k|k-1}),\; P_k = (I - K_k H)\,P_{k|k-1} \tag{2.5}$$

In DeepSORT and ByteTrack, the standard deviations in Q and R are proportional to the box height through two weights σ_p (position) and σ_v (velocity). These weights directly control how much the tracker trusts its motion prediction versus the new detection, and are therefore important tunable parameters.

2.3 Data Association

The affinity between track i and detection j can be the IoU of the predicted and detected boxes, the Mahalanobis distance $d^{(1)}$ under the Kalman covariance, or the cosine distance $d^{(2)}$ between appearance embeddings. DeepSORT combines them as:

$$c_{ij} = \lambda\, d^{(1)}(i,j) + (1-\lambda)\, d^{(2)}(i,j) \tag{2.6}$$

The assignment minimising the total cost is found with the Hungarian algorithm in $O(n^3)$ time, and matches whose cost exceeds a threshold are rejected.

2.3.1 ByteTrack two-stage association

ByteTrack splits detections into high-score (score > τ_{high}) and low-score (τ_{low} < score ≤ τ_{high}) sets. High-score detections are associated first with all tracks; the remaining tracks are then associated with low-score detections using IoU only. Unmatched high-score detections with score > τ_{new} start new tracks; tracks unmatched for more than a buffer of B frames are deleted.

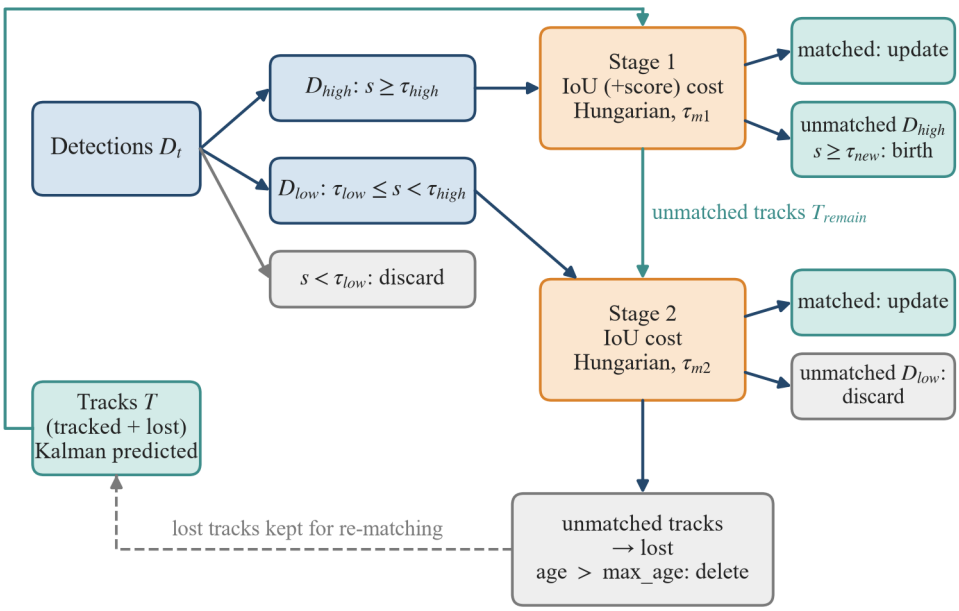


Figure 2.2 ByteTrack two-stage association: high-score detections are matched with all tracks first, low-score detections with the remaining tracks; unmatched high-score detections create new tracks and unmatched tracks become lost or are deleted.

2.4 Tracking Evaluation Metrics

$$MOTA = 1 - \Sigma_t (FN_t + FP_t + IDSW_t) / \Sigma_t GT_t \quad (2.7)$$

$$IDF1 = 2\,IDTP / (2\,IDTP + IDFP + IDFN) \quad (2.8)$$

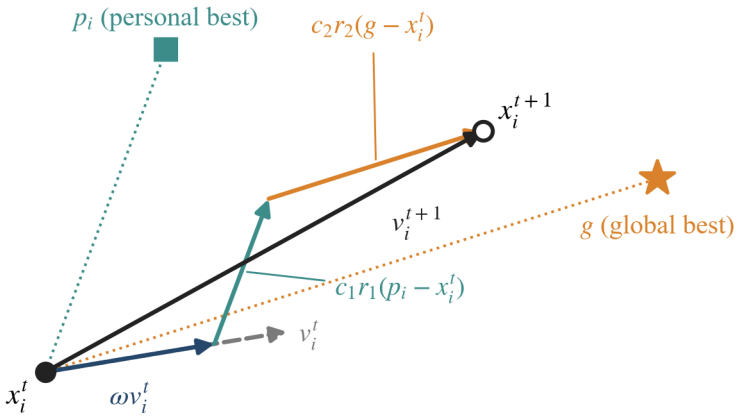
$$HOTA_\alpha = sqrt(DetA_\alpha \cdot AssA_\alpha),\ HOTA = (1/19)\,\Sigma_\alpha HOTA_\alpha \quad (2.9)$$

MOTA is dominated by detection errors, IDF1 by identity consistency, while HOTA balances detection accuracy (DetA) and association accuracy (AssA), averaged over 19 localisation thresholds α from 0.05 to 0.95.

2.5 Intelligent Optimization Methods

2.5.1 Particle swarm optimization

In PSO, each particle i has a position x_i (a candidate solution) and velocity v_i. It remembers its personal best p_i, and the swarm shares the global best g:



$$v_i^{t+1} = \omega v_i^t + c_1 r_1 (p_i - x_i^t) + c_2 r_2 (g - x_i^t), \quad x_i^{t+1} = x_i^t + v_i^{t+1}, \quad r_1, r_2 \sim U(0, 1)$$

Figure 2.3 Vector illustration of the PSO velocity update as the sum of inertia, cognitive and social components.

$$v_i^{t+1} = \omega\, v_i^t + c_1\, r_1\, (p_i - x_i^t) + c_2\, r_2\, (g - x_i^t) \quad (2.10)$$

$$x_i^{t+1} = x_i^t + v_i^{t+1} \quad (2.11)$$

where ω is the inertia weight, c_1 and c_2 are cognitive and social acceleration coefficients, and r_1, r_2 are uniform random numbers in [0, 1].

2.5.2 Genetic algorithm

GA maintains a population of chromosomes and iteratively applies selection (e.g., tournament), crossover (e.g., simulated binary crossover) and mutation (e.g., polynomial mutation), with elitism preserving the best individuals.

2.5.3 Grey wolf optimizer

GWO ranks the three best solutions as α, β and δ wolves, which guide the rest of the population. The control parameter a decreases linearly from 2 to 0, shifting the search from exploration to exploitation.

2.5.4 Bayesian optimization

Bayesian optimization builds a probabilistic surrogate of the objective (Gaussian process or TPE) and chooses the next point by maximising an acquisition function such as expected improvement. It is sample-efficient but sequential and scales poorly with dimension and parallelism.

2.6 Chapter Summary

This chapter introduced the tracking-by-detection framework, the Kalman filter, data association, tracking metrics and the intelligent optimization algorithms that form the basis of this thesis.

Chapter 3
Problem Formulation and Parameter Analysis

3.1 Baseline Tracker

The baseline tracker used throughout this thesis is a ByteTrack-style tracker with a Kalman filter, two-stage IoU association, and an optional appearance branch (an OSNet re-identification model) for the first association stage. This tracker is chosen because it is widely used, strong on the MOT benchmarks and has a moderate number of interpretable parameters. A YOLOX detector trained on the MOT17 training set (as released by the ByteTrack authors) is used as the fixed detector.

3.2 Parameter Space

Symbol	Meaning	Type	Range	Default
τ_{high}	High detection score threshold	continuous	[0.3, 0.8]	0.6
τ_{low}	Low detection score threshold	continuous	[0.05, 0.3]	0.1
τ_{new}	New-track score threshold	continuous	[0.4, 0.9]	0.7
θ_1	First-stage match threshold (1 − IoU)	continuous	[0.5, 0.95]	0.8
θ_2	Second-stage match threshold	continuous	[0.3, 0.8]	0.5
B	Track buffer (frames)	integer	[10, 120]	30
σ_p	Kalman position noise weight	continuous	[0.01, 0.2]	1/20
σ_v	Kalman velocity noise weight	continuous	[0.001, 0.05]	1/160
λ	Motion–appearance weight	continuous	[0, 1]	0.98
n_{min}	Minimum box area (pixels)	integer	[0, 400]	10

Table 3.1 Tunable parameters of the baseline tracker. Defaults follow the public ByteTrack and DeepSORT implementations; please confirm them against the code version you use.

3.3 Mechanical Role of Each Parameter

To interpret sensitivity results and optimised values, it is necessary to understand exactly where each parameter acts in the per-frame processing loop. The baseline tracker processes frame t in the following order, and each parameter is attached to one step.

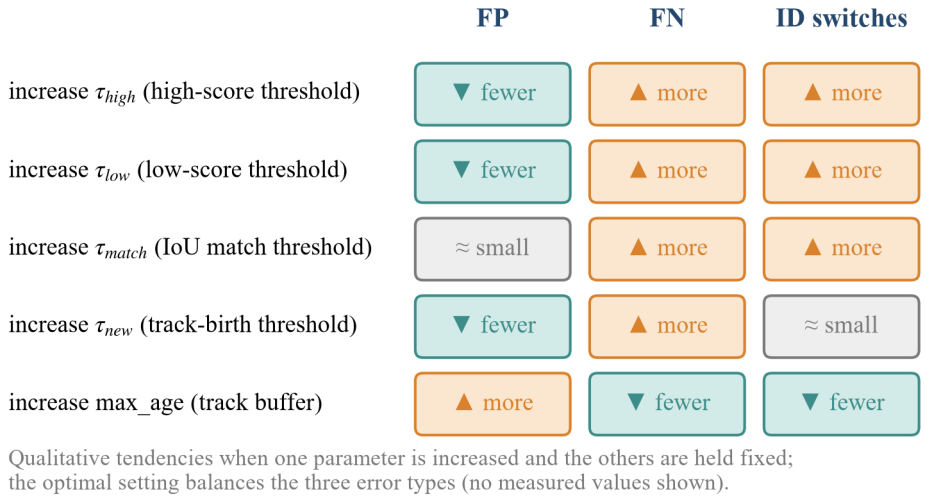


Figure 3.1 Conceptual schematic of how increasing each ByteTrack parameter tends to affect false positives, false negatives and identity switches (qualitative tendencies, not measured data).

3.3.1 Detection filtering: n_{min}, τ_{low} and τ_{high}

Cached detections are first filtered by box area: boxes smaller than n_{min} pixels, and in the public implementation boxes with an extreme aspect ratio, are discarded. This removes tiny spurious boxes but also removes distant pedestrians, so n_{min} mainly trades false positives against false negatives for small objects. The remaining detections are partitioned by score: $D_{high} = \{ d : s(d) > \tau_{high} \}$ and $D_{low} = \{ d : \tau_{low} < s(d) \leq \tau_{high} \}$; detections with $s(d) \leq \tau_{low}$ are discarded entirely. Raising τ_{high} moves detections from the first to the second association stage, where they can only extend existing tracks and can never start new ones; lowering τ_{low} lets more weak detections extend tracks through occlusion but also lets background clutter attach to tracks whose prediction overlaps it.

3.3.2 Motion prediction: σ_p and σ_v

Every active and lost track is propagated by the Kalman prediction step. In the ByteTrack implementation the process noise covariance Q is diagonal with standard deviations σ_p h for position and size components and σ_v h for velocity components, where h is the current box height, and the measurement noise R uses σ_p h as well. A larger σ_p inflates both Q and R; a larger σ_v lets the velocity estimate change more quickly. The ratio between process and measurement noise determines the Kalman gain: when Q is large relative to R the filter follows detections closely and adapts to camera motion, and when Q is small the filter smooths over detection jitter but lags behind abrupt motion. Because IoU-based association compares the predicted box with

detections, the noise weights influence association only through the predicted mean, which is why their effect is most visible for long occlusions and for moving cameras.

3.3.3 First association stage: θ_1 and λ

Active and lost tracks are associated with D_{high}. The cost matrix is 1 − IoU, optionally fused with the appearance cosine distance through λ (Equation 2.6) and multiplied by detection score in the "fuse score" variant. The Hungarian algorithm returns the minimum-cost assignment, and any matched pair whose cost exceeds θ_1 is rejected. θ_1 is therefore the maximum allowed dissimilarity: $\theta_1 = 0.8$ accepts a match whenever the IoU exceeds 0.2. Increasing θ_1 allows matches under fast motion or poor prediction, but also allows identity swaps when two people cross. λ controls how much the first stage relies on appearance: λ near 1 makes the stage purely motion-based, while smaller values make it robust to motion errors but sensitive to appearance similarity between people dressed alike.

3.3.4 Second association stage: θ_2

Tracks that remain unmatched and were active in the previous frame are associated with D_{low} by IoU only, rejecting pairs with cost above θ_2. Because low-score detections are less reliable, θ_2 is typically stricter than θ_1. This stage is the main mechanism through which ByteTrack recovers occluded objects; its effect on MOTA is large in crowded scenes, where many true objects are detected with low confidence.

3.3.5 Track birth: τ_{new}

Unmatched high-score detections are first matched with unconfirmed tracks (tracks seen in only one frame); remaining detections with score above τ_{new} start new tentative tracks. A high τ_{new} suppresses tracks born from false positives but delays the start of tracks for objects entering the scene with low confidence, which increases false negatives at the beginning of trajectories. Since τ_{new} only applies to detections already in D_{high}, the effective birth threshold is $\max(\tau_{high}, \tau_{new})$, which motivates the ordering constraint below.

3.3.6 Track death: B

Tracks that remain unmatched are marked lost and kept for B frames (in the public implementation B is scaled by frame rate divided by 30). Lost tracks still participate in the first association stage, so B determines the longest occlusion across which an identity can be preserved. If B is too short, re-appearing objects receive new identities (fragmentation, lower IDF1 and AssA); if B is too long, a lost track whose prediction has drifted may capture a different object (identity switch). Because the Kalman prediction drifts linearly with time, B interacts strongly with σ_v and θ_1: a long buffer is only useful if the prediction remains close enough for the IoU to pass the threshold.

3.3.7 Summary of interactions

Parameter	Step affected	Increase tends to	Main metric affected	Strongest interaction
n_{min}	Detection filtering	Fewer FP, more FN for small objects	DetA, MOTA	τ_{high}
τ_{high}	Score partition	Fewer FP in stage 1, fewer births	DetA, MOTA	τ_{new}, θ_2
τ_{low}	Score partition	Fewer weak detections used	DetA (recall)	θ_2
σ_p, σ_v	Kalman prediction	Faster adaptation, noisier prediction	AssA, IDSW	B, θ_1
θ_1	Stage-1 gating	More permissive matching	AssA, IDSW	λ, B
λ	Stage-1 cost	More reliance on motion	AssA, IDF1	θ_1
θ_2	Stage-2 gating	More occlusion recovery	DetA, MOTA	τ_{low}
τ_{new}	Track birth	Fewer spurious tracks, later births	DetA, IDF1	τ_{high}
B	Track deletion	Longer identity memory	IDF1, AssA, IDSW	σ_v, θ_1

Table 3.2 Qualitative effect of each tracker parameter, derived from the processing logic. The direction of each effect is an expectation to be verified by the sensitivity analysis.

Constraints: $\tau_{low} < \tau_{high} \leq \tau_{new}$. Constraints are handled by repair (projection to the feasible region) rather than by penalty, because infeasible configurations cannot be evaluated meaningfully.

3.4 Optimization Model

Let $\theta \in \Theta$ denote the parameter vector, V the set of validation sequences and Track(θ, v) the output of the tracker on sequence v. The single-objective problem is:

$$\theta^* = arg\ max_{\theta \in \Theta} f(\theta) = (1/|V|) \Sigma_{v \in V} HOTA(\ Track(\theta, v),\ GT_v\ ) \qquad (3.1)$$

The objective has the following characteristics, which drive the design of the optimizer:

- **Black-box and non-differentiable**: thresholds and the Hungarian assignment create a piecewise-constant landscape.
- **Expensive**: one evaluation processes thousands of frames.

- **Mixed-variable**: integer parameters such as B.
- **Multimodal with plateaus**: many configurations give nearly identical scores.
- **Risk of over-fitting**: parameters can over-fit the validation sequences, so generalisation must be tested on held-out sequences.

3.5 Data Split for Fair Optimization

The MOT17 training set (7 sequences) is split by sequence. Following the common "half" protocol, the first half of each training sequence is used for optimization and the second half for validation. In addition, a leave-one-sequence-out protocol is used to evaluate cross-scene generalisation. The MOT17 and MOT20 test sets are used only for final submission to the evaluation server, if submission is possible, never for tuning.

3.6 Parameter Sensitivity Analysis

Before optimization, a global sensitivity analysis is carried out to understand which parameters matter:

- **One-at-a-time (OAT) analysis**: vary each parameter across its range while keeping the others at default, and plot HOTA, MOTA, IDF1 and ID switches (Figure 3.1).
- **Sobol variance-based analysis**: using Saltelli sampling (e.g. with the SALib library), compute first-order indices S_i and total-order indices S_{Ti}. A large gap $S_{Ti} - S_i$ indicates strong interactions.

For a model output $Y = f(\theta)$ with independent inputs, the Sobol decomposition attributes the output variance to individual parameters and their interactions. The first-order and total-order indices of parameter i are:

$$S_i = V_{\theta_i}(E_{\theta_{\sim i}}[Y \mid \theta_i]) / V(Y), \; S_{Ti} = E_{\theta_{\sim i}}(V_{\theta_i}[Y \mid \theta_{\sim i}]) / V(Y) \qquad (3.2)$$

where $\theta_{\sim i}$ denotes all parameters except θ_i. S_i measures the fraction of output variance explained by θ_i alone and S_{Ti} the fraction explained by θ_i including all its interactions. With Saltelli's scheme [29], the indices are estimated from $N_s(D + 2)$ model evaluations, where N_s is the base sample size; with $D = 10$ and $N_s = 64$ this requires 768 tracker runs, which is comparable to one optimisation run and is affordable with detection caching. Bootstrap confidence intervals are reported for each index, and indices whose interval includes zero are treated as negligible. Because the ordering constraint makes the threshold parameters dependent, the sensitivity analysis samples the three thresholds in the unconstrained box and applies the repair operator inside the model, so that the analysis measures the sensitivity of the repaired tracker.

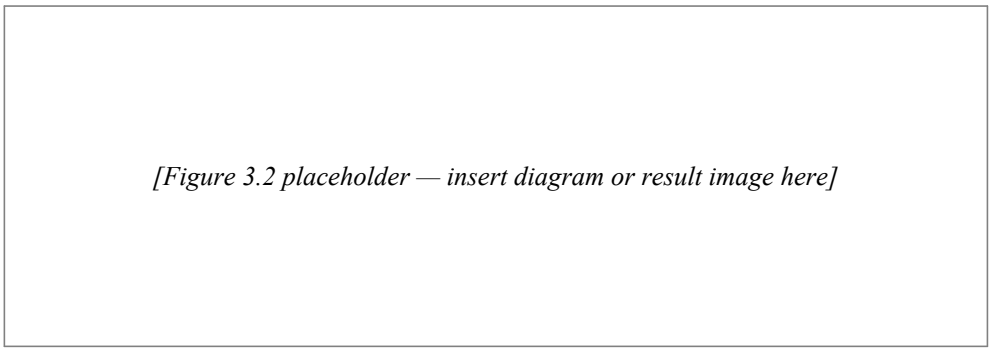


Figure 3.2 One-at-a-time sensitivity of HOTA to each tracker parameter on the MOT17 half-validation split.

Parameter	First-order S_i	Total-order S_{Ti}	Rank
τ_{high}	[ ]	[ ]	[ ]
τ_{new}	[ ]	[ ]	[ ]
θ_l	[ ]	[ ]	[ ]
B	[ ]	[ ]	[ ]
σ_p	[ ]	[ ]	[ ]
...	[ ]	[ ]	[ ]

Table 3.3 Sobol sensitivity indices of the tracker parameters (to be computed from your experiments).

The expected outcome, based on the role of each parameter, is that detection thresholds and the first-stage matching threshold dominate HOTA, the track buffer mainly affects ID switches and IDF1, and the Kalman noise weights matter more for sequences with camera motion. The sensitivity results are used to fix insensitive parameters at their defaults, reducing the dimension of the search space.

3.7 Chapter Summary

This chapter defined the baseline tracker, its parameter space and the optimization model, and described the sensitivity analysis used to understand and reduce the search space.

Chapter 4
Improved PSO for Tracker Parameter Optimization

4.1 Limitations of Standard PSO

Standard PSO has three weaknesses on this problem: (1) random initialisation may cover the search space unevenly when the population is small, which it must be because evaluations are expensive; (2) a constant or linearly decreasing inertia weight does not adapt to the search state; (3) on plateau-rich, multimodal landscapes the swarm tends to collapse prematurely onto a local optimum.

4.2 Chaotic Initialisation

The logistic chaotic map generates sequences that are ergodic and non-repeating:

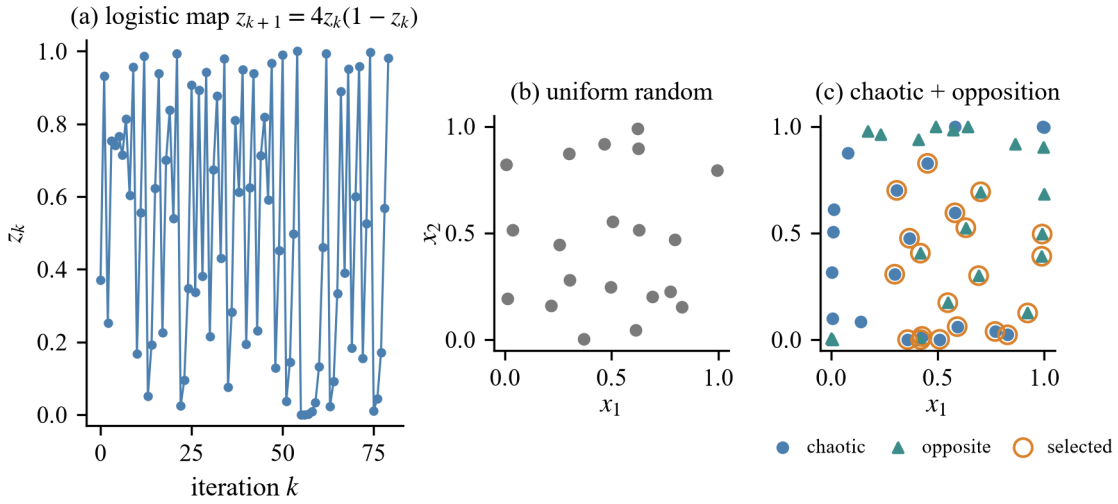


Figure 4.1 (a) Logistic-map sequence; (b) uniform random initialisation; (c) chaotic initialisation combined with opposition-based learning and selection of the best candidates (illustrative, synthetic data).

$$z_{k+1} = \mu z_k (1 - z_k),\ \mu = 4,\ z_0 \notin \{0,\ 0.25,\ 0.5,\ 0.75,\ 1\} \tag{4.1}$$

For each dimension d, the chaotic value is mapped to the parameter range: $x_{id} = lb_d + z_{id}(ub_d - lb_d)$. In addition, the default parameter vector is inserted as one particle, so the optimized result is guaranteed to be at least as good as the default on the optimization data. Opposition-based learning is used to generate the opposite of each initial particle, and the better half of the combined set forms the initial swarm.

4.3 Nonlinear Adaptive Inertia Weight

The inertia weight decreases nonlinearly from $\omega_{max} = 0.9$ to $\omega_{min} = 0.4$:

$$\omega(t) = \omega_{min} + (\omega_{max} - \omega_{min}) \cdot \cos(\pi t / (2T)) \quad (4.2)$$

This keeps ω large for longer than a linear schedule, encouraging exploration early, and decreases it quickly near the end for fine exploitation.

4.4 Asynchronous Acceleration Coefficients

The cognitive coefficient decreases and the social coefficient increases over iterations:

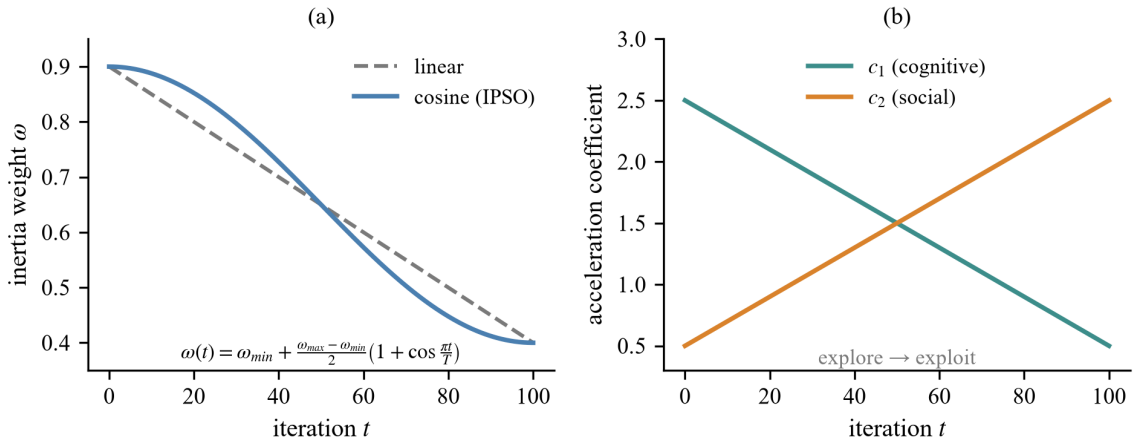


Figure 4.2 (a) Linear versus cosine-decreasing inertia weight ω; (b) asynchronous schedules of the cognitive (c1) and social (c2) coefficients over the iterations.

$$c_1(t) = 2.5 - 2.0\, t / T,\ c_2(t) = 0.5 + 2.0\, t / T \quad (4.3)$$

so that particles first explore around their own experience and later converge toward the swarm's best solution.

4.5 Lévy-Flight Mutation of the Global Best

When the global best has not improved for s consecutive iterations (stagnation), a Lévy-flight perturbation is applied to it:

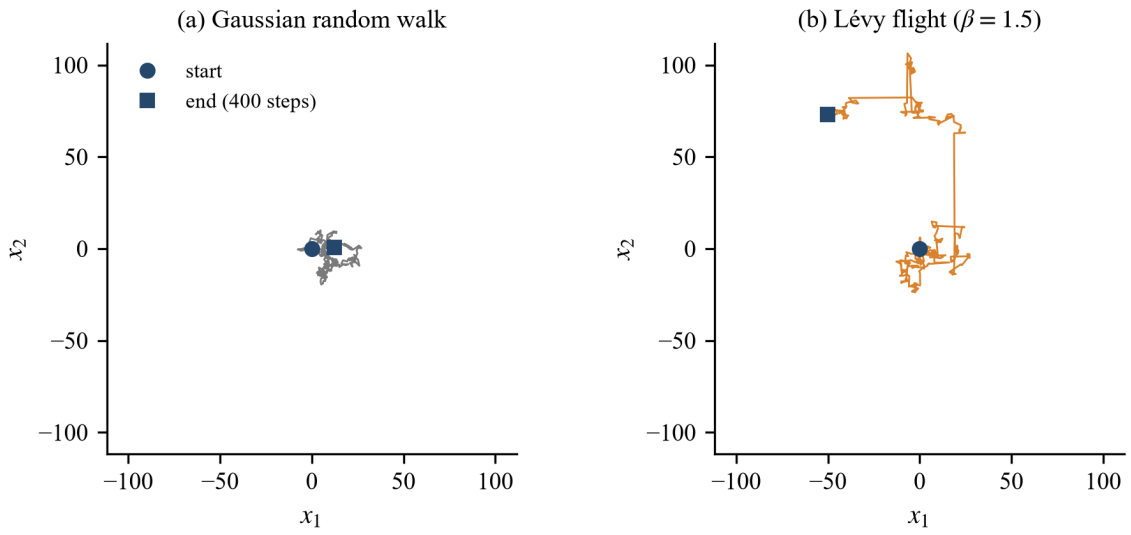


Figure 4.3 Comparison of a Gaussian random walk and a Lévy flight (β = 1.5, Mantegna algorithm) with the same number of steps, showing occasional long jumps (illustrative, synthetic data).

$$g' = g + \alpha \cdot L(\beta) \odot (ub - lb),\ L(\beta) = u / |v|^{1/\beta} \tag{4.4}$$

where $u \sim N(0, \sigma_u^2)$, $v \sim N(0, 1)$, $\beta = 1.5$, $\odot$ denotes element-wise multiplication and α is a step scale ($\alpha = 0.01$ to 0.05 of the range). Following Mantegna [27], σ_u is chosen so that $u / |v|^{1/\beta}$ approximates a symmetric Lévy-stable distribution with index β:

$$\sigma_u = \{ \Gamma(1+\beta) \sin(\pi\beta/2) / [\Gamma((1+\beta)/2) \cdot \beta \cdot 2^{(\beta-1)/2}] \}^{1/\beta},\ \sigma_v = 1 \tag{4.5}$$

For $\beta = 1.5$ this gives $\sigma_u \approx 0.6966$. (The symbol σ_v here denotes the standard deviation of v and should not be confused with the Kalman velocity weight.) The resulting step lengths have a heavy power-law tail, $P(|L| > s) \propto s^{-\beta}$, so most perturbations are local refinements while occasional long jumps carry the global best to a distant region of the search space. The mutated solution replaces g only if it is better by more than the tolerance ε (greedy selection), and up to $m_L = 3$ mutation trials are made per stagnation event, so that the cost of mutation is bounded.

4.6 Convergence and Stability Analysis of PSO

The choice of ω, c_1 and c_2 determines whether the swarm converges, oscillates or diverges. Following the analyses of Clerc and Kennedy [44] and Trelea [45], consider one dimension of one particle and assume that p and g are fixed. Replacing the random factors by their expectations, $\varphi_1 = c_1 r_1$, $\varphi_2 = c_2 r_2$ and $\varphi = \varphi_1 + \varphi_2$, the update can be written as a second-order linear recurrence:

$$x^{t+1} = (1 + \omega - \varphi) x^t - \omega x^{t-1} + \varphi_1 p + \varphi_2 g \tag{4.6}$$

The fixed point is the weighted attractor $x^* = (\varphi_1 p + \varphi_2 g) / \varphi$. The deviation from x^* evolves according to the characteristic equation:

$$\lambda^2 - (1 + \omega - \varphi)\,\lambda + \omega = 0 \tag{4.7}$$

The deterministic system converges to x* if and only if both roots satisfy $|\lambda| < 1$, which yields the stability region:

$$0 < \varphi < 2\,(1 + \omega),\ -1 < \omega < 1 \tag{4.8}$$

When the roots are complex, i.e. when $(1 + \omega - \varphi)^2 < 4\omega$, the particle oscillates around the attractor with a decay factor sqrt(ω) per iteration, which produces a useful sampling of the region between p and g. With the random factors uniformly distributed, φ has expectation $(c_1 + c_2) / 2$, and second-order (variance) stability analyses under the stagnation assumption give the tighter, widely cited condition:

$$c_1 + c_2 < 24\,(1 - \omega^2) / (7 - 5\omega) \tag{4.9}$$

For ω = 0.7298 this bound is approximately 3.04 and is satisfied by the constriction setting $c_1 = c_2 = 1.49618$. For the IPSO schedules of Equations 4.2 and 4.3, $c_1(t) + c_2(t) = 3.0$ at every iteration, while ω(t) decreases from 0.9 to 0.4. The bound equals approximately 1.75 at ω = 0.9 and 2.90 at ω = 0.4; therefore the early iterations lie outside the second-order stable region, deliberately producing growing oscillation (exploration), and the swarm enters a region close to the boundary only at the end of the run. To prevent divergence in the early phase, velocities are clamped to $v_{max,d} = \kappa\,(ub_d - lb_d)$ with κ = 0.2, and boundary reflection (Section 4.7) limits positions. The ablation in Chapter 6 compares this schedule with a schedule that remains inside the stable region throughout, so that the benefit of early instability can be assessed empirically rather than assumed.

[Figure 4.4 placeholder — insert diagram or result image here]

Figure 4.4 Stability regions of PSO in the $(\omega, c_1 + c_2)$ plane, showing the first-order and second-order boundaries and the trajectory of the IPSO parameter schedule over iterations.

These results assume a stagnant p and g and ignore the coupling between particles, so they describe the behaviour of the search rather than guaranteeing convergence to a global optimum. Standard PSO is not guaranteed to converge even to a local optimum; the Lévy-flight mutation and the re-evaluation of top candidates in IPSO do not change this theoretical fact, and the claims of this thesis are therefore empirical.

4.7 Handling Integer and Constrained Parameters

4.7.1 Integer parameters

The track buffer B and the minimum area n_{min} are integers. They are optimised as continuous variables and rounded to the nearest integer only when the tracker is called, while the swarm keeps the continuous position. This preserves small velocity components that would be destroyed by rounding the position itself, and lets particles drift across integer boundaries over several iterations. Because rounding maps many continuous positions to the same configuration, an evaluation cache keyed by the rounded and quantised parameter vector is maintained: if a particle maps to a configuration that has already been evaluated, the stored value is reused at no cost. Continuous thresholds are quantised to a resolution of 0.005 for the purpose of this cache only, which is below the resolution at which tracking metrics change measurably.

4.7.2 Ordering constraint

The constraint $\tau_{low} < \tau_{high} \leq \tau_{new}$ is handled by repair. Given a raw position, the three thresholds are sorted in ascending order and assigned to τ_{low}, τ_{high} and τ_{new}; if $\tau_{high} - \tau_{low}$ is below a minimum gap $\delta = 0.02$, τ_{low} is decreased by δ. Each value is then clipped to its own range. The repaired position is written back to the particle (Lamarckian repair), so that the swarm memory remains feasible. An alternative is Baldwinian repair, in which the repaired vector is evaluated but the particle keeps its unrepaired position; it preserves diversity but makes the personal best inconsistent with the evaluated configuration, and is therefore not used. Penalty functions are not appropriate here because an infeasible configuration has no meaningful tracking score from which to subtract a penalty.

4.7.3 Bound handling

If a position component leaves $[lb_d, ub_d]$, it is reflected: $x_d \leftarrow 2\, ub_d - x_d$ above the upper bound and $x_d \leftarrow 2\, lb_d - x_d$ below the lower bound, followed by clipping if the reflected value is still outside. The velocity component is reversed and halved, $v_d \leftarrow -0.5\, v_d$. Reflection is preferred over absorption (clipping to the bound), because absorption accumulates particles on the boundary and, as many tracking parameters have their optimum in the interior, produces wasted evaluations at extreme values.

4.8 Efficient Objective Evaluation

4.8.1 Detection caching

The detector is the most expensive part of the tracker, but its output does not depend on the tracker parameters, except for the score thresholds, which are applied after detection. Therefore, detections (boxes, scores and optional appearance embeddings) for all frames

are computed once with a very low score threshold (0.01) and stored on disk. Each objective evaluation only re-runs association, which is typically one to two orders of magnitude faster than running the full pipeline. Non-maximum suppression is applied once at caching time with the default IoU threshold and is not part of the search space, because changing it would invalidate the cache. Appearance embeddings are computed for all cached boxes, which is affordable because the re-identification network is run only once.

4.8.2 Parallel evaluation design

The evaluation of one swarm iteration is organised as a pool of independent tasks. A task is a pair (configuration, sequence), so that an iteration of N particles on |V| sequences produces up to N|V| tasks. A master process holds the swarm state, generates and repairs configurations, consults the evaluation cache and submits the remaining tasks to a process pool of P workers. Each worker loads the cached detections of its sequence (memory-mapped, so that sequences are shared between workers rather than copied), runs association and writes the tracker output to an in-memory buffer, then computes the per-sequence HOTA components with TrackEval. The master aggregates per-sequence components into the dataset-level metrics — HOTA must be aggregated from the per-sequence detection and association counts rather than averaged, if the combined score is to match the official evaluation — and only then updates the swarm. Because sequences differ greatly in length, tasks are submitted longest-first, which reduces idle time at the end of each iteration (a standard longest-processing-time scheduling heuristic). The update is synchronous: the swarm waits for all particles before updating, which keeps the algorithm identical to sequential PSO and makes runs reproducible given the seed. Asynchronous updating would keep workers busier but makes the result depend on timing, which complicates the statistical comparison.

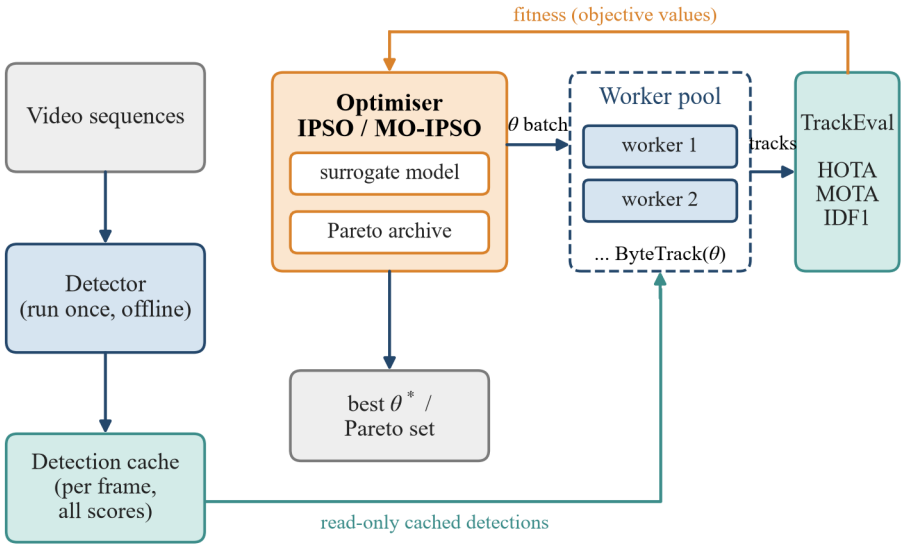


Figure 4.5 Optimisation framework: the IPSO/MO-IPSO optimiser dispatches parameter candidates to parallel workers that run association on cached detections, and TrackEval returns fitness values.

4.8.3 Surrogate model and pre-screening

The surrogate approximates f(θ) from the archive of all evaluated pairs $A = \{(\theta_j, f_j)\}$. Two models are considered.

- **Random forest** [63]: an ensemble of M = 100 regression trees, each trained on a bootstrap sample of A with a random subset of features considered at each split. The prediction is the mean of the tree predictions, μ(θ), and the spread between trees, s(θ), serves as an uncertainty estimate, as in SMAC [56]. Random forests handle integer parameters and plateaus naturally because they are piecewise constant, they train in milliseconds for the archive sizes considered here (at most a few hundred points), and they require almost no tuning.
- **Gaussian process** [64]: a zero-mean GP with a Matérn 5/2 kernel with automatic relevance determination (one length-scale per parameter), on parameters normalised to [0, 1] and standardised objective values, with a noise term to model evaluation noise. Kernel hyper-parameters are fitted by maximising the marginal likelihood. The GP provides calibrated uncertainty and its fitted length-scales offer a second view of parameter relevance, complementary to the Sobol analysis.

At iteration t, for each particle i, the standard velocity update is applied k = 5 times with independent random numbers r_1, r_2, producing k candidate positions. Each candidate is scored with the lower-confidence-bound style acquisition adapted to maximisation:

$$a(\theta) = \mu(\theta) + \kappa_a \, s(\theta) \tag{4.10}$$

with $\kappa_a = 1.0$, and the candidate with the largest $a(\theta)$ is evaluated with the true objective. Expected improvement, $EI(\theta) = (\mu - f^+)\,\Phi(Z) + s\,\varphi(Z)$ with $Z = (\mu - f^+) / s$, where f^+ is the best observed value and Φ and φ are the standard normal cdf and pdf, is evaluated as an alternative criterion. Because the candidates are all genuine PSO moves, pre-screening only biases the random component of the swarm towards promising regions and does not replace the PSO dynamics. Pre-screening is disabled during the first two iterations, when the archive contains fewer than 2N points and the surrogate is unreliable, and the surrogate is refitted after every iteration.

4.8.4 Noise-robust selection

Strictly speaking, for fixed cached detections the tracker is deterministic, so the objective is not stochastic. It is, however, noisy in the sense relevant to optimisation: tiny parameter changes cause discrete changes in individual association decisions, producing fluctuations of a few hundredths of a HOTA point that do not reflect a true improvement and do not generalise. IPSO uses three mechanisms to avoid chasing such fluctuations.

- **Tolerance-based update.** A personal best p_i is replaced only if $f(x_i) > f(p_i) + \varepsilon$, and the global best likewise. With $\varepsilon = 0.05$ HOTA points, the swarm does not move its attractors on spurious gains.
- **Smoothed fitness for ranking.** When the surrogate is available, the value used to rank candidates for re-evaluation is the average of the observed value and the surrogate prediction at that point, which shrinks isolated outliers towards their neighbourhood.
- **Final re-evaluation and selection.** At the end of the run, the K = 5 best distinct configurations in the archive are evaluated on the validation split, and the final configuration is the one with the best validation score. If two configurations differ by less than ε, the one closer to the default parameter vector (in normalised Euclidean distance) is chosen, a conservative rule that favours configurations that are less likely to have over-fitted.

4.9 Algorithm Procedure

The overall procedure is summarised in the following steps.

- Step 1: Pre-compute and cache detections for all optimization sequences.
- Step 2: Initialise N particles with logistic chaotic mapping and opposition-based learning; insert the default parameters.
- Step 3: Evaluate all particles in parallel; initialise personal bests and the global best.

- Step 4: For t = 1 ... T: update ω, c_1, c_2; update velocities and positions; repair constraints; pre-screen with the surrogate; evaluate; update bests with tolerance ε.
- Step 5: If the global best stagnates for s iterations, apply Lévy-flight mutation.
- Step 6: Return the global best; re-evaluate top candidates on the validation split.

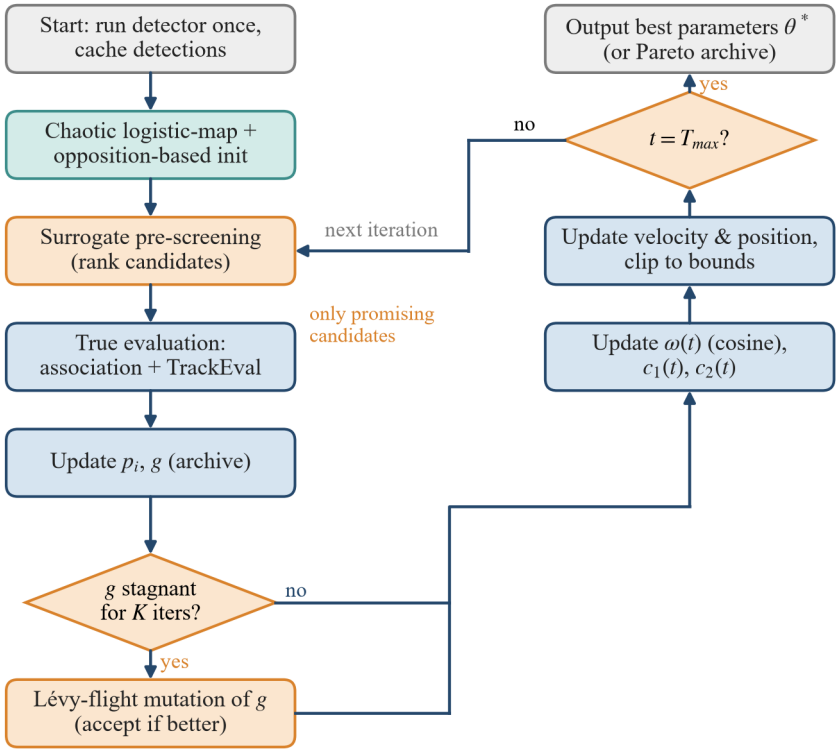


Figure 4.6 Flowchart of IPSO including detection caching, chaotic/opposition-based initialisation, surrogate pre-screening, stagnation-triggered Lévy mutation and adaptive parameter updates.

4.10 Detailed IPSO Pseudocode

The complete algorithm is given below as numbered steps. Inputs are the search space bounds lb, ub, the default vector θ_0, the optimisation sequences V_{opt}, the validation sequences V_{val}, population size N, iteration count T, stagnation limit s, tolerance ε, number of surrogate candidates k and number of final candidates K.

- 1. Run the detector once on every frame of V_{opt} and V_{val} with score threshold 0.01; store boxes, scores and embeddings. Initialise the evaluation archive $A \leftarrow \emptyset$ and the cache $C \leftarrow \emptyset$.
- 2. Draw z_0 uniformly in (0, 1) excluding the fixed points of the logistic map; generate $N \times D$ chaotic values by $z_{k+1} = 4\, z_k(1 - z_k)$ and map them to $x_{id} = lb_d + z_{id}(ub_d - lb_d)$.
- 3. For each chaotic particle compute its opposite $\tilde{x}_{id} = lb_d + ub_d - x_{id}$. Repair all 2N vectors and add θ_0.

- 4. Evaluate the 2N + 1 vectors in parallel (using C to skip duplicates); add results to A. Keep the best N, ensuring θ_0 is retained, as the initial swarm. Set $p_i \leftarrow x_i$, $g \leftarrow \arg\max f(p_i)$, and initialise velocities v_{id} uniformly in $[-v_{max,d}, v_{max,d}]$. Set the stagnation counter $c_s \leftarrow 0$.
- 5. For t = 1 to T, repeat Steps 6 to 12.
- 6. Compute $\omega(t)$, $c_1(t)$ and $c_2(t)$ with Equations 4.2 and 4.3.
- 7. If t > 2, fit the surrogate on A.
- 8. For each particle i: if the surrogate is fitted, generate k candidate velocities with independent r_1, r_2; otherwise generate one. Clamp each velocity to $\pm v_{max}$, compute the candidate position, apply bound reflection and constraint repair, and select the candidate with the largest acquisition value. Set v_i and x_i to the selected candidate.
- 9. Evaluate all new positions in parallel (cache lookup first); append to A.
- 10. For each i, if $f(x_i) > f(p_i) + \varepsilon$ then $p_i \leftarrow x_i$. If $\max f(p_i) > f(g) + \varepsilon$, update g and reset $c_s \leftarrow 0$; otherwise $c_s \leftarrow c_s + 1$.
- 11. If $c_s \geq s$: for up to m_L trials, generate g' by Lévy flight (Equation 4.4 with σ_u from Equation 4.5), repair and evaluate it; if $f(g') > f(g) + \varepsilon$, set $g \leftarrow g'$, replace the worst personal best by g', and stop the trials. Reset $c_s \leftarrow 0$.
- 12. Record the best-so-far value, swarm diversity and the number of true evaluations used; stop early if the evaluation budget is exhausted.
- 13. Select the K best distinct configurations from A, evaluate them on V_{val}, and return the best according to the tie-breaking rule of Section 4.8.

Evaluations used in Steps 4, 9 and 11 are all counted towards the common budget, so that IPSO is compared with the other optimisers at exactly the same number of true tracker runs. Cache hits are not counted, but their number is reported.

4.11 Complexity

The cost is dominated by objective evaluations: $N \times (T + 1)$ evaluations, each costing $O(F \cdot n^3)$ for F frames and n objects per frame due to the Hungarian algorithm. With detection caching and P parallel workers, wall-clock time is approximately $N(T + 1) \cdot t_{assoc} / P$. With N = 20 and T = 30, 620 evaluations are required, which is feasible within hours on a multi-core workstation.

The overhead of IPSO itself is small in comparison. Chaotic initialisation and opposition learning cost O(ND). The surrogate adds $O(M\,|A|\log|A| \cdot D)$ per iteration for the random forest, or $O(|A|^3)$ for the GP; with $|A| \leq 700$ this is at most a few seconds per iteration, negligible compared with the minutes required for N tracker evaluations.

Memory is dominated by the detection cache, which for MOT17 with embeddings occupies on the order of a few gigabytes and is shared between workers.

4.12 Chapter Summary

This chapter proposed the IPSO algorithm, combining chaotic opposition-based initialisation, nonlinear inertia, asynchronous acceleration coefficients and Lévy-flight mutation, and an efficient evaluation framework based on detection caching, parallelism and surrogate pre-screening.

Chapter 5
Multi-Objective Parameter Optimization

5.1 Motivation

Single-objective optimization of MOTA tends to reduce false negatives at the cost of more identity switches, while optimizing IDF1 may sacrifice detection recall. Different applications prefer different trade-offs: crowd counting values detection accuracy, while person re-identification across cameras values identity consistency. A multi-objective formulation provides the user with a set of Pareto-optimal parameter configurations.

5.2 Formulation

$$max_{\theta} F(\theta) = [\, DetA(\theta), AssA(\theta) \,] \text{ or } [\, HOTA(\theta), -IDSW(\theta) \,] \quad (5.1)$$

A configuration θ_a dominates θ_b if it is no worse in all objectives and strictly better in at least one. The goal is to approximate the Pareto front.

5.3 Multi-Objective IPSO

The IPSO is extended into MO-IPSO following the MOPSO framework [22]:

- An external archive stores non-dominated solutions, with crowding distance used to prune the archive when it is full.
- Each particle selects its leader from the archive by roulette selection favouring less crowded regions, which spreads the swarm along the front.
- Chaotic initialisation and Lévy-flight mutation are retained; mutation is applied to archive members chosen at random.

NSGA-II [21] is used as the comparison baseline. Hypervolume (HV) and the number of non-dominated solutions are used to compare the Pareto fronts. The components are detailed in the following subsections.

5.3.1 Objective normalisation

The two objectives have different units and ranges: HOTA lies in [0, 100] and changes by a few points over the search space, while IDSW is an integer count that may vary by hundreds. All crowding, leader-selection and hypervolume computations are performed on normalised objectives. Both objectives are converted to minimisation, $f_1 = -\text{HOTA}$ and $f_2 = \text{IDSW}$, and each is scaled by fixed bounds obtained from the initial population, $\tilde{f}_m = (f_m - f_m^{min}) / (f_m^{max} - f_m^{min})$. Fixed bounds are used, rather than bounds updated every iteration, so that the hypervolume values reported at different iterations remain comparable.

5.3.2 Archive maintenance

The archive A_{nd} has a maximum size $N_A = 50$. After each iteration, every newly evaluated particle is offered to the archive:

- if the new solution is dominated by any archive member, it is rejected;
- otherwise, it is inserted and all archive members it dominates are removed;
- if two solutions have identical (rounded) parameter vectors or objective vectors equal within the tolerance ε in every objective, only the earlier one is kept, which prevents the archive from filling with near-duplicates produced by plateaus;
- if the archive exceeds N_A, the crowding distance of all members is computed and the member with the smallest crowding distance is removed; the crowding distances are recomputed after each removal, until the size limit is respected.

Removing one member at a time and recomputing, rather than truncating in a single step, avoids removing two neighbouring solutions that were only crowded with respect to each other.

5.3.3 Crowding distance

For a set of non-dominated solutions sorted by the m-th objective, with solutions indexed 1 ... L in that order, the crowding distance of solution j is [21]:

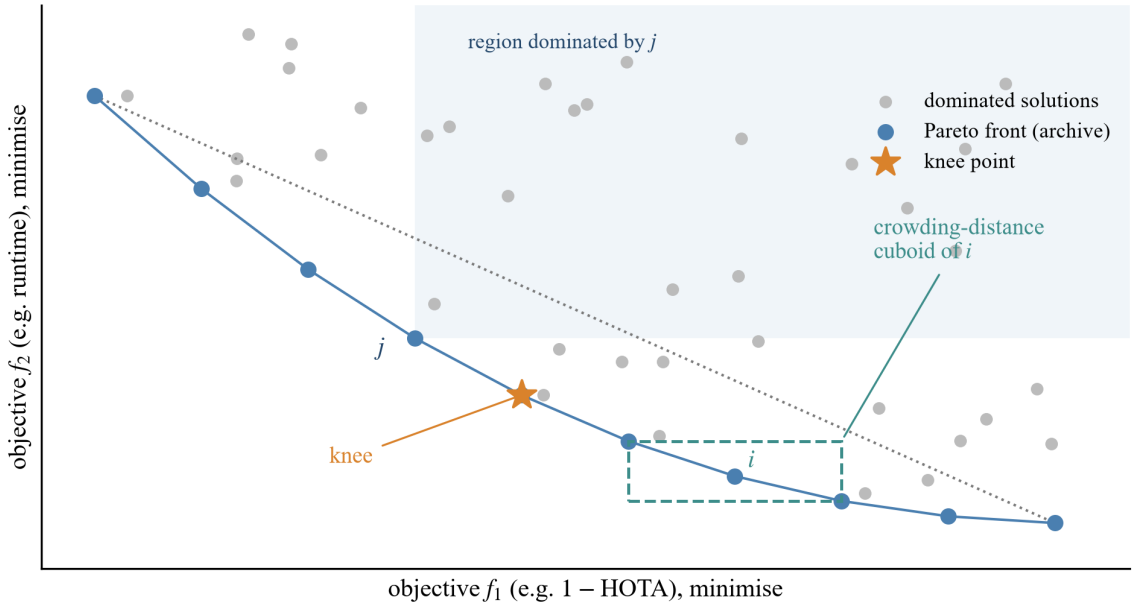


Figure 5.1 Pareto-optimality concepts: non-dominated front stored in the archive, dominated solutions, the crowding-distance cuboid and the knee point (illustrative, synthetic data).

$$CD_j = \Sigma_{m=1}^{M} \left(f_m(j+1) - f_m(j-1) \right) / \left(f_m^{max} - f_m^{min} \right) \quad (5.2)$$

with $CD_1 = CD_L = \infty$ for the boundary solutions of each objective, so that the extreme points of the front are always kept. For two objectives, CD_j is proportional to the

perimeter of the rectangle formed by the two neighbours of solution j, and therefore measures how isolated it is along the front.

5.3.4 Leader selection

Each particle requires a social attractor in place of the global best g. The leader of particle i at iteration t is drawn from the archive by binary tournament on crowding distance: two archive members are drawn uniformly at random and the one with the larger crowding distance is chosen, with the extreme solutions (infinite distance) treated as the most preferred. This is a simpler alternative to the adaptive hypercube grid of the original MOPSO [22], with the same intent: particles are guided towards sparsely populated regions of the front, which improves spread. The personal best p_i is updated if the new position dominates it; if neither dominates the other, the new position replaces p_i with probability 0.5.

5.3.5 Mutation and stagnation in the multi-objective setting

Since there is no single global best, stagnation is defined by the hypervolume of the archive: if HV has not improved by more than a relative tolerance of 0.1% for s iterations, Lévy-flight mutation is applied to three archive members chosen by the same crowding tournament, and the mutated solutions are offered to the archive. The nonlinear inertia weight and time-varying acceleration coefficients are kept unchanged from the single-objective algorithm.

5.3.6 Hypervolume indicator

The hypervolume of a set S of normalised objective vectors with respect to a reference point r that is dominated by all of them is the Lebesgue measure of the region dominated by S and bounded by r [67]:

$$HV(S) = \Lambda(\cup_{s \in S} [s_1, r_1] \times [s_2, r_2] \times \dots \times [s_M, r_M]) \tag{5.3}$$

where Λ denotes the Lebesgue measure. A larger HV indicates a front that is both closer to the true Pareto front and better spread. For two objectives, HV is computed exactly by sorting the points by f_1 and summing the areas of the rectangles between consecutive points and the reference point, in O(L log L) time. The reference point is set to r = (1.1, 1.1) in normalised coordinates, slightly worse than the worst values observed in the initial population, and the same r is used for all algorithms and runs. Because hypervolume is Pareto-compliant, a front with higher HV cannot be dominated by a front with lower HV. In addition to HV, the inverted generational distance with respect to the union of all non-dominated solutions found by all algorithms in all runs (a reference front) is reported, together with the number of distinct non-dominated configurations.

5.4 Decision Making

A final configuration is selected from the front by the knee-point method (the point with maximum distance to the line connecting the extreme solutions) or according to application-specific weights.

5.4.1 Knee-point selection

A knee is a solution at which a small improvement in one objective requires a large sacrifice in the other [66]. In normalised objective space, let e_1 and e_2 be the two extreme solutions of the front (best HOTA and fewest IDSW). The knee is the solution j that maximises the perpendicular distance to the line through e_1 and e_2:

$$d_j = | (e_{2,2} - e_{1,2}) f_1(j) - (e_{2,1} - e_{1,1}) f_2(j) + e_{2,1} e_{1,2} - e_{2,2} e_{1,1} | / || e_2 - e_1 || \quad (5.4)$$

Only solutions on the convex side of the line (that is, dominating some point of the line segment) are eligible. If the front is nearly linear, so that no solution has a distance above a small threshold, no knee exists and the user should choose by weights instead.

5.4.2 Preference-based selection

When the application has explicit preferences, a weighted Tchebycheff scalarisation is used: the selected configuration minimises $\max_m w_m | f_m - z_m^* |$, where z^* is the ideal point (the best value of each objective in the archive) and w is a user-specified weight vector. Unlike a weighted sum, the Tchebycheff function can select solutions in non-convex parts of the front. Finally, as in the single-objective case, the selected configuration and its neighbours on the front are re-evaluated on the validation split before being adopted.

[Figure 5.2 placeholder — insert diagram or result image here]

Figure 5.2 Example Pareto front of HOTA versus ID switches obtained by MO-IPSO and NSGA-II, with the knee point and the extreme solutions marked.

5.5 Chapter Summary

This chapter extended IPSO to multi-objective optimization, giving users a set of trade-off configurations between detection quality and identity stability.

Chapter 6
Experiments and Analysis

Note to the author: this chapter specifies the experimental design. Cells marked [ ] must be filled with the values from your own runs. Report the mean and standard deviation over at least 5 independent optimizer runs with different random seeds, because all compared optimizers are stochastic.

6.1 Experimental Setup

6.1.1 Datasets

- **MOT17**: 7 training and 7 test sequences of pedestrians, static and moving cameras; the half split described in Chapter 3 is used for optimization and validation.
- **MOT20**: 4 training and 4 test sequences of extremely crowded scenes; used to test whether optimal parameters differ in dense crowds.
- Optionally, **DanceTrack**, with uniform appearance and complex motion, to test association-dominated scenarios.

6.1.2 Environment and settings

Item	Setting
Hardware	[GPU model], [CPU cores], [RAM]
Detector	YOLOX-X (ByteTrack MOT17 weights), fixed
Evaluation	TrackEval (HOTA, MOTA, IDF1, IDSW)
Budget	620 objective evaluations per optimizer (N = 20, T = 30)
Repeats	5 independent runs per optimizer

Table 6.1 Experimental settings.

6.1.3 Compared methods

Default parameters; grid search (coarse, same budget); random search; GA; DE; standard PSO (linearly decreasing ω); GWO; CMA-ES; Bayesian optimization (TPE via Optuna); random-forest Bayesian optimization (RF-BO); and the proposed IPSO. All optimizers use the same search space, the same evaluation budget and the same cached detections.

To ensure a fair comparison, every optimiser is configured with settings recommended in its original publication or in a widely used implementation, and none is tuned on the tracking task itself. The settings are listed in Table 6.2. Grid search uses the largest lattice that fits in the budget over the parameters ranked most important by the Sobol

analysis, with the others at default values. CMA-ES [53] is included as an additional strong continuous baseline, and SMAC-style random-forest BO [56] as a model-based baseline whose surrogate matches the one used by IPSO, which isolates the contribution of the swarm dynamics from that of the surrogate.

Optimiser	**Population / batch**	**Main settings**	**Source of settings**
Random search	20 per batch	Uniform sampling in the box	[24]
GA	20	Tournament size 2, SBX (η_c = 15, p_c = 0.9), polynomial mutation (η_m = 20, p_m = 1/D), elitism 1	[18, 21]
DE	20	DE/rand/1/bin, F = 0.5, CR = 0.9	[19]
PSO	20	ω linear 0.9 → 0.4, $c_1 = c_2$ = 2.0, v_{max} = 0.2 range	[16, 17]
GWO	20	a linear 2 → 0	[20]
CMA-ES	4 + ⌊3 ln D⌋ default, restarts allowed	σ_0 = 0.3 (normalised space)	[53]
TPE	1 (sequential), 10 random start points	Optuna default sampler	[25, 26]
RF-BO (SMAC-style)	1 (sequential)	100 trees, EI acquisition	[56]
IPSO (ours)	20	Chapter 4 settings, s = 3, ε = 0.05, k = 5	This thesis

Table 6.2 Settings of all compared optimisers. All use the same search space, repair operator, evaluation budget (620 tracker runs) and cached detections.

Sequential model-based methods (TPE and RF-BO) cannot exploit the P parallel workers as effectively as population methods. Two budgets are therefore reported: equal number of evaluations (the primary comparison, which measures sample efficiency) and equal wall-clock time (which measures practical efficiency on a parallel machine).

6.2 Single-Objective Optimization Results

Method	**HOTA**	**MOTA**	**IDF1**	**IDSW**	**Time (h)**
Default	[ ]	[ ]	[ ]	[ ]	–

Method	HOTA	MOTA	IDF1	IDSW	Time (h)
Grid search	[ ]	[ ]	[ ]	[ ]	[ ]
Random search	[ ]	[ ]	[ ]	[ ]	[ ]
GA	[ ]	[ ]	[ ]	[ ]	[ ]
DE	[ ]	[ ]	[ ]	[ ]	[ ]
PSO	[ ]	[ ]	[ ]	[ ]	[ ]
GWO	[ ]	[ ]	[ ]	[ ]	[ ]
CMA-ES	[ ]	[ ]	[ ]	[ ]	[ ]
Bayesian (TPE)	[ ]	[ ]	[ ]	[ ]	[ ]
RF-BO	[ ]	[ ]	[ ]	[ ]	[ ]
IPSO (ours)	[ ]	[ ]	[ ]	[ ]	[ ]

Table 6.3 Tracking results on the MOT17 half-validation split with optimized parameters (mean ± std over 5 runs).

Whether the difference between IPSO and each baseline is statistically significant ($p < 0.05$) is determined with the protocol of Section 6.3.

6.2.1 Per-sequence results

Dataset-level averages can hide that an optimiser improves some sequences while degrading others. Table 6.4 reports HOTA per sequence on the MOT17 half-validation split for the default parameters and for the parameters found by the best run of each of the three strongest optimisers. Sequences recorded with a moving camera (MOT17-05, -10, -11, -13) are expected to respond differently from static-camera sequences (MOT17-02, -04, -09), which should be discussed in relation to the Kalman noise parameters.

Sequence	Camera	Default	TPE	PSO	IPSO
MOT17-02	static	[ ]	[ ]	[ ]	[ ]
MOT17-04	static	[ ]	[ ]	[ ]	[ ]
MOT17-05	moving	[ ]	[ ]	[ ]	[ ]
MOT17-09	static	[ ]	[ ]	[ ]	[ ]
MOT17-10	moving	[ ]	[ ]	[ ]	[ ]
MOT17-11	moving	[ ]	[ ]	[ ]	[ ]
MOT17-13	moving	[ ]	[ ]	[ ]	[ ]
Combined	–	[ ]	[ ]	[ ]	[ ]

Table 6.4 Per-sequence HOTA on the MOT17 half-validation split. Columns other than Default use the parameters selected by each optimiser. Replace TPE and PSO by the strongest baselines in your results if different.

Sequence	DetA (default)	DetA (IPSO)	AssA (default)	AssA (IPSO)	IDSW (default)	IDSW (IPSO)
MOT17-02	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
MOT17-04	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
MOT17-05	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
MOT17-09	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
MOT17-10	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
MOT17-11	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
MOT17-13	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Table 6.5 Decomposition of the per-sequence change into detection accuracy, association accuracy and identity switches.

6.2.2 Runtime and evaluation cost

Method	True evaluations	Cache hits	Optimiser overhead (s)	Wall-clock (h), P workers	Wall-clock to reach 99% of final HOTA (h)
Random search	[ ]	[ ]	[ ]	[ ]	[ ]
GA	[ ]	[ ]	[ ]	[ ]	[ ]
DE	[ ]	[ ]	[ ]	[ ]	[ ]
PSO	[ ]	[ ]	[ ]	[ ]	[ ]
GWO	[ ]	[ ]	[ ]	[ ]	[ ]
CMA-ES	[ ]	[ ]	[ ]	[ ]	[ ]
TPE	[ ]	[ ]	[ ]	[ ]	[ ]
RF-BO	[ ]	[ ]	[ ]	[ ]	[ ]
IPSO	[ ]	[ ]	[ ]	[ ]	[ ]

Table 6.6 Computational cost of each optimiser (mean over 5 runs). Optimiser overhead excludes tracker evaluation time.

Pipeline stage	Full pipeline per evaluation (s)	With detection caching (s)	Speed-up
Detection	[ ]	0 (cached)	–
Re-identification embedding	[ ]	0 (cached)	–
Association and track management	[ ]	[ ]	–
Metric computation (TrackEval)	[ ]	[ ]	–
Total	[ ]	[ ]	[ ]

Table 6.7 Time per objective evaluation on the MOT17 half-train split with and without detection caching.

6.3 Statistical Testing Protocol

All compared optimisers are stochastic, and the differences between well-tuned configurations may be small. The following protocol is used to decide whether reported differences are meaningful.

- **Pairwise comparison with IPSO.** For each baseline, the final validation HOTA values of the R = 5 runs are compared with those of IPSO using the Wilcoxon rank-sum (Mann–Whitney U) test, because runs of different optimisers are independent. (A signed-rank test is appropriate only for paired samples, for example when the same seed controls an identical initial population, or when comparing per-sequence results of two fixed configurations.) With only five runs per method the smallest attainable two-sided p-value of the rank-sum test is about 0.008, so a significant result requires near-complete separation; if the budget permits, R = 10 runs are recommended.
- **Multiple comparisons.** Because IPSO is compared with eight baselines, the Holm–Bonferroni correction is applied to the family of p-values.
- **Ranking across problems.** For the comparison across datasets and benchmark functions, the Friedman test on average ranks followed by the Nemenyi post-hoc test is used, as recommended by Demšar [65], and the result is shown as a critical-difference diagram.
- **Effect size.** Since significance alone does not indicate practical relevance, the Vargha–Delaney A_{12} effect size (the probability that a random IPSO run outperforms a random baseline run) and the difference of medians are reported.
- **Per-sequence paired test.** To assess whether the selected IPSO configuration improves over the default configuration consistently across sequences, the Wilcoxon signed-rank test is applied to the paired per-sequence HOTA values of Table 6.4, keeping in mind the small number of sequences.

Baseline vs. IPSO	**Median HOTA (baseline)**	**Median HOTA (IPSO)**	**p (rank-sum)**	**p (Holm)**	**A_{12}**	**Significant**
Random search	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
GA	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
DE	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
PSO	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
GWO	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
CMA-ES	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
TPE	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
RF-BO	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Table 6.8 Statistical comparison between IPSO and each baseline on validation HOTA (MOT17 half split).

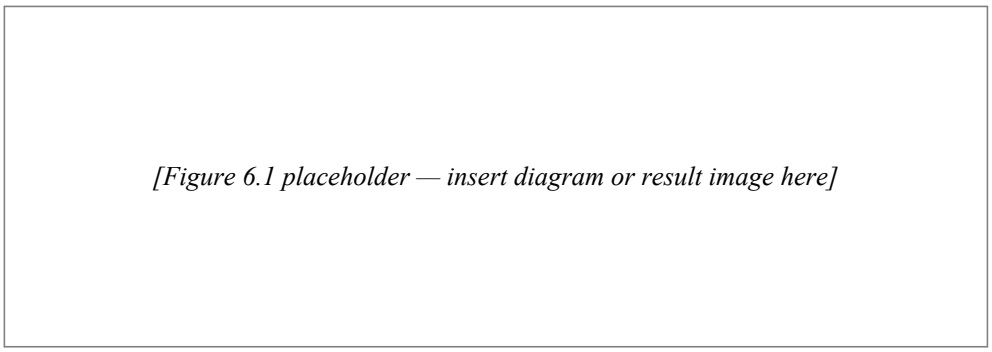


Figure 6.1 Critical-difference diagram of the average ranks of all optimisers over datasets and benchmark functions (Friedman and Nemenyi tests, $\alpha = 0.05$).

6.4 Convergence Analysis

Plot the best-so-far HOTA against the number of evaluations for all optimizers, averaged over runs with shaded standard deviation (Figure 6.1). Discuss convergence speed (evaluations to reach 99% of the final value) and final quality. Also show the swarm diversity over iterations for PSO and IPSO to demonstrate the effect of chaotic initialisation and Lévy-flight mutation on premature convergence.

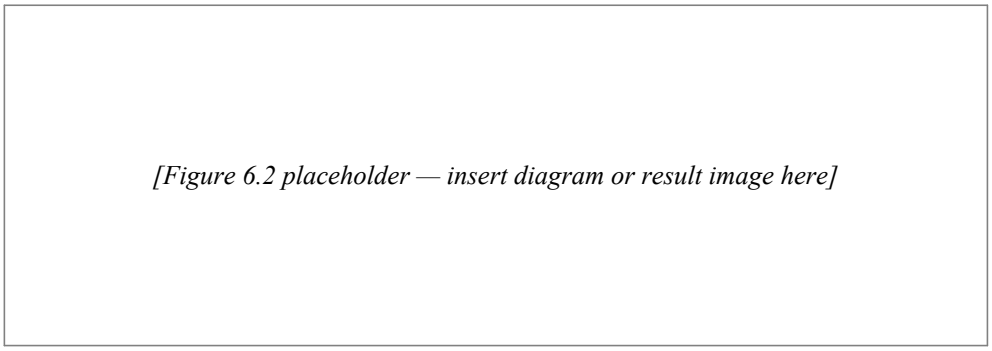


Figure 6.2 Convergence curves of all optimizers (best-so-far HOTA versus number of evaluations).

6.5 Benchmark Function Validation

To validate IPSO independently of tracking, it should also be tested on standard benchmark functions (e.g., Sphere, Rastrigin, Ackley, Griewank, Rosenbrock in 10 dimensions) against PSO, GA and GWO with the same budget.

Function	Definition	Search range	Global minimum	Character
Sphere	$\Sigma\, x_i^2$	$[-100, 100]$	0	Unimodal, separable
Rastrigin	$\Sigma\,(x_i^2 - 10\cos(2\pi x_i) + 10)$	$[-5.12, 5.12]$	0	Highly multimodal, regular

Function	Definition	Search range	Global minimum	Character
Ackley	$-20\exp(-0.2\,\mathrm{sqrt}(\Sigma\, x_i^2/D)) - \exp(\Sigma\cos(2\pi\, x_i)/D) + 20 + e$	$[-32, 32]$	0	Multimodal, nearly flat outer region
Griewank	$\Sigma\, x_i^2/4000 - \Pi\cos(x_i/\mathrm{sqrt}(i)) + 1$	$[-600, 600]$	0	Multimodal, non-separable
Rosenbrock	$\Sigma\,(\,100\,(x_{i+1} - x_i^2)^2 + (x_i - 1)^2\,)$	$[-30, 30]$	0	Narrow curved valley

Table 6.9 Benchmark functions used for validation (D = 10). Because several of these functions have their optimum at the origin, which some swarm algorithms are biased towards, shifted versions (optimum moved to a random point) are also evaluated.

Function	PSO	GA	GWO	IPSO
Sphere	[ ]	[ ]	[ ]	[ ]
Rastrigin	[ ]	[ ]	[ ]	[ ]
Ackley	[ ]	[ ]	[ ]	[ ]
Griewank	[ ]	[ ]	[ ]	[ ]
Rosenbrock	[ ]	[ ]	[ ]	[ ]

Table 6.10 Mean best fitness on benchmark functions (10-D, 30 runs).

6.6 Ablation Study of IPSO

Variant	Chaotic init.	Nonlinear ω	Adaptive c_1, c_2	Lévy mutation	HOTA
PSO					[ ]
+ chaotic init	✓				[ ]
+ nonlinear ω	✓	✓			[ ]
+ adaptive c	✓	✓	✓		[ ]
IPSO (full)	✓	✓	✓	✓	[ ]

Table 6.11 Ablation of IPSO components.

A second ablation evaluates the efficiency strategies: with versus without detection caching (wall-clock time), and with versus without surrogate pre-screening (HOTA at equal budget).

Variant	Surrogate	Candidates k	Caching	HOTA at 620 evaluations	Wall-clock (h)
IPSO without surrogate	none	1	yes	[ ]	[ ]

Variant	Surrogate	Candidates k	Caching	HOTA at 620 evaluations	Wall-clock (h)
IPSO + RF surrogate (UCB)	RF	5	yes	[ ]	[ ]
IPSO + RF surrogate (EI)	RF	5	yes	[ ]	[ ]
IPSO + GP surrogate (EI)	GP	5	yes	[ ]	[ ]
IPSO without caching	RF	5	no	[ ]	[ ]

Table 6.12 Ablation of the evaluation-efficiency strategies.

Variant	Parameter schedule	Tolerance ε	Final re-evaluation	Validation HOTA	Held-out HOTA
IPSO-stable	$\omega = 0.7298$, $c_1 = c_2 = 1.496$ (constriction)	0.05	yes	[ ]	[ ]
IPSO-TVAC	Chapter 4 schedules	0	no	[ ]	[ ]
IPSO-TVAC + ε	Chapter 4 schedules	0.05	no	[ ]	[ ]
IPSO (full)	Chapter 4 schedules	0.05	yes	[ ]	[ ]

Table 6.13 Ablation of the parameter schedule and of the noise-robust selection mechanisms. The gap between validation and held-out HOTA indicates over-fitting.

6.7 Generalisation Experiments

- **Within-dataset**: parameters optimized on the first half of MOT17 sequences are evaluated on the second half.
- **Leave-one-sequence-out**: optimize on six sequences, test on the seventh; repeat for all seven.
- **Cross-dataset**: parameters optimized on MOT17 are tested on MOT20, and vice versa, compared with parameters optimized directly on the target.
- **Test server**: if available, submit the best configuration to MOTChallenge.

Train → Test	Default HOTA	Optimized HOTA	Gain
MOT17-half → MOT17-val	[ ]	[ ]	[ ]
MOT17 → MOT20	[ ]	[ ]	[ ]
MOT20 → MOT17	[ ]	[ ]	[ ]

Table 6.14 Generalisation of optimized parameters.

Held-out sequence	Default HOTA	IPSO HOTA (tuned on other six)	IPSO HOTA (tuned on itself, oracle)	Gain over default
MOT17-02	[ ]	[ ]	[ ]	[ ]
MOT17-04	[ ]	[ ]	[ ]	[ ]
MOT17-05	[ ]	[ ]	[ ]	[ ]
MOT17-09	[ ]	[ ]	[ ]	[ ]
MOT17-10	[ ]	[ ]	[ ]	[ ]
MOT17-11	[ ]	[ ]	[ ]	[ ]
MOT17-13	[ ]	[ ]	[ ]	[ ]
Mean	[ ]	[ ]	[ ]	[ ]

Table 6.15 Leave-one-sequence-out generalisation on MOT17. The oracle column is an upper bound that indicates how much of the per-scene optimum is captured by parameters tuned on other scenes.

6.8 Multi-Objective Results

Report the hypervolume and Pareto fronts of MO-IPSO and NSGA-II (Figure 5.1 style), and analyse the parameter values along the front, for example how the track buffer B and the match threshold θ_1 change between the high-DetA end and the high-AssA end.

Algorithm	HV (mean ± std)	IGD	No. of non-dominated solutions	HOTA at knee	IDSW at knee
NSGA-II	[ ]	[ ]	[ ]	[ ]	[ ]
MOPSO	[ ]	[ ]	[ ]	[ ]	[ ]
MO-IPSO (ours)	[ ]	[ ]	[ ]	[ ]	[ ]

Table 6.16 Multi-objective results on the MOT17 half split (HOTA versus IDSW, 5 runs, equal budget of 620 evaluations).

6.9 Discussion of Optimized Parameters

Tabulate the optimized parameters for MOT17 and MOT20 and interpret them physically. For instance, in crowded MOT20 scenes the optimizer may prefer stricter matching thresholds and higher new-track thresholds to avoid identity confusion, whereas in MOT17 sequences with camera motion it may increase the Kalman position noise. These interpretations must be stated only after they are confirmed by your results.

Parameter	Default	Grid	Random	GA	PSO	GWO	TPE	IPSO
τ_{high}	0.6	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
τ_{low}	0.1	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
τ_{new}	0.7	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
θ_1	0.8	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Parameter	Default	Grid	Random	GA	PSO	GWO	TPE	IPSO
θ_2	0.5	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
B	30	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
σ_p	1/20	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
σ_v	1/160	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
λ	0.98	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
n_{min}	10	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Table 6.17 Parameter values selected by each optimiser on the MOT17 half split (best run of each optimiser). DE, CMA-ES and RF-BO columns may be added in the same form.

Parameter	MOT17 (IPSO)	MOT20 (IPSO)	Std. over 5 IPSO runs (MOT17)	Std. over 5 IPSO runs (MOT20)
τ_{high}	[ ]	[ ]	[ ]	[ ]
τ_{new}	[ ]	[ ]	[ ]	[ ]
θ_1	[ ]	[ ]	[ ]	[ ]
θ_2	[ ]	[ ]	[ ]	[ ]
B	[ ]	[ ]	[ ]	[ ]
σ_p	[ ]	[ ]	[ ]	[ ]

Table 6.18 Comparison of IPSO-optimised parameters between MOT17 and MOT20, with the run-to-run standard deviation of each value.

The run-to-run standard deviation of each optimised parameter is informative in its own right. A parameter that varies widely between runs while the final HOTA is nearly constant lies on a plateau and is effectively insensitive; this should be consistent with a small total-order Sobol index. A parameter whose optimised value is stable across runs but differs between MOT17 and MOT20 is a genuinely scene-dependent parameter and is the most important candidate for the scene-adaptive tuning discussed in Chapter 7.

6.10 Threats to Validity

- **Internal validity.** All optimisers share the same implementation of the tracker, repair operator, cache and metric code, so differences cannot be attributed to implementation details of the objective. The settings of the baselines follow their original publications; it remains possible that a baseline would perform better with settings tuned for small budgets, which should be acknowledged when interpreting the comparison.
- **Construct validity.** HOTA is used as the single objective; conclusions about "better" parameters are therefore conclusions about HOTA. MOTA and IDF1 are reported alongside so that a gain in HOTA obtained at the cost of another metric is visible.

- **External validity.** Results are obtained with one detector and one tracker family. Because optimal thresholds depend on the score calibration of the detector, the optimised values themselves are not expected to transfer to a different detector; the claim that is expected to transfer is the relative efficiency of the optimisers.
- **Over-fitting to validation data.** Repeatedly selecting configurations on the same validation split biases the reported validation score upwards. For this reason the final numbers are reported on held-out sequences (leave-one-sequence-out and cross-dataset protocols), and validation scores are used only for selection.

6.11 Chapter Summary

This chapter designed the experiments to evaluate IPSO against baseline optimizers, analyse convergence, validate on benchmark functions, ablate each component, test generalisation and evaluate the multi-objective extension.

Chapter 7
Conclusion and Future Work

7.1 Conclusions

- Parameter tuning of tracking-by-detection algorithms was formulated as an expensive, noisy, mixed-variable black-box optimization problem, and a sensitivity analysis method was designed to identify the parameters that matter most.
- An improved PSO algorithm was proposed, combining chaotic and opposition-based initialisation, nonlinear inertia weight, asynchronous acceleration coefficients and Lévy-flight mutation of the global best.
- An efficient evaluation framework based on detection caching, parallel evaluation and surrogate pre-screening was designed, making population-based optimization practical.
- A multi-objective extension was proposed to provide Pareto-optimal trade-offs between detection accuracy and identity stability.
- A rigorous experimental protocol was designed, including comparisons with six optimizers, statistical testing, ablations and generalisation tests.

7.2 Limitations

- The detector is fixed; joint optimization of detector thresholds and tracker parameters, or of training hyper-parameters, is not considered.
- Parameters are static for a whole sequence; optimal values may change over time within a video.
- The risk of over-fitting to the validation sequences remains when the validation set is small.

7.3 Future Work

- Online, scene-adaptive parameter adjustment, for example using reinforcement learning to change thresholds frame by frame according to crowd density.
- Extending the method to transformer-based trackers and to multi-camera tracking.
- Meta-learning a mapping from scene descriptors (density, camera motion, frame rate) to good initial parameters, to warm-start the optimizer in new scenes.

References

[1] Bewley A, Ge Z, Ott L, Ramos F, Upcroft B. Simple online and realtime tracking. IEEE ICIP, 2016.

[2] Wojke N, Bewley A, Paulus D. Simple online and realtime tracking with a deep association metric. IEEE ICIP, 2017.

[3] Zhang Y, Sun P, Jiang Y, et al. ByteTrack: Multi-object tracking by associating every detection box. ECCV, 2022.

[4] Cao J, Pang J, Weng X, Khirodkar R, Kitani K. Observation-centric SORT: Rethinking SORT for robust multi-object tracking. CVPR, 2023.

[5] Aharon N, Orfaig R, Bobrovsky B Z. BoT-SORT: Robust associations multi-pedestrian tracking. arXiv:2206.14651, 2022.

[6] Du Y, Zhao Z, Song Y, et al. StrongSORT: Make DeepSORT great again. IEEE Transactions on Multimedia, 2023.

[7] Zhang Y, Wang C, Wang X, Zeng W, Liu W. FairMOT: On the fairness of detection and re-identification in multiple object tracking. IJCV, 2021.

[8] Meinhardt T, Kirillov A, Leal-Taixé L, Feichtenhofer C. TrackFormer: Multi-object tracking with transformers. CVPR, 2022.

[9] Bernardin K, Stiefelhagen R. Evaluating multiple object tracking performance: The CLEAR MOT metrics. EURASIP Journal on Image and Video Processing, 2008.

[10] Ristani E, Solera F, Zou R, Cucchiara R, Tomasi C. Performance measures and a data set for multi-target, multi-camera tracking. ECCV Workshops, 2016.

[11] Luiten J, Osep A, Dendorfer P, et al. HOTA: A higher order metric for evaluating multi-object tracking. IJCV, 2021.

[12] Milan A, Leal-Taixé L, Reid I, Roth S, Schindler K. MOT16: A benchmark for multi-object tracking. arXiv:1603.00831, 2016.

[13] Dendorfer P, Rezatofighi H, Milan A, et al. MOT20: A benchmark for multi object tracking in crowded scenes. arXiv:2003.09003, 2020.

[14] Kalman R E. A new approach to linear filtering and prediction problems. Journal of Basic Engineering, 1960.

[15] Kuhn H W. The Hungarian method for the assignment problem. Naval Research Logistics Quarterly, 1955.

[16] Kennedy J, Eberhart R. Particle swarm optimization. Proc. IEEE International Conference on Neural Networks, 1995.

[17] Shi Y, Eberhart R. A modified particle swarm optimizer. IEEE World Congress on Computational Intelligence, 1998.

[18] Holland J H. Adaptation in Natural and Artificial Systems. University of Michigan Press, 1975.

[19] Storn R, Price K. Differential evolution — a simple and efficient heuristic for global optimization over continuous spaces. Journal of Global Optimization, 1997.

[20] Mirjalili S, Mirjalili S M, Lewis A. Grey wolf optimizer. Advances in Engineering Software, 2014.

[21] Deb K, Pratap A, Agarwal S, Meyarivan T. A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation, 2002.

[22] Coello C A C, Pulido G T, Lechuga M S. Handling multiple objectives with particle swarm optimization. IEEE Transactions on Evolutionary Computation, 2004.

[23] Wolpert D H, Macready W G. No free lunch theorems for optimization. IEEE Transactions on Evolutionary Computation, 1997.

[24] Bergstra J, Bengio Y. Random search for hyper-parameter optimization. JMLR, 2012.

[25] Bergstra J, Bardenet R, Bengio Y, Kégl B. Algorithms for hyper-parameter optimization. NeurIPS, 2011.

[26] Akiba T, Sano S, Yanase T, Ohta T, Koyama M. Optuna: A next-generation hyperparameter optimization framework. KDD, 2019.

[27] Mantegna R N. Fast, accurate algorithm for numerical simulation of Lévy stable stochastic processes. Physical Review E, 1994.

[28] Tizhoosh H R. Opposition-based learning: A new scheme for machine intelligence. CIMCA, 2005.

[29] Saltelli A. Making best use of model evaluations to compute sensitivity indices. Computer Physics Communications, 2002.

[30] Ge Z, Liu S, Wang F, Li Z, Sun J. YOLOX: Exceeding YOLO series in 2021. arXiv:2107.08430, 2021.

[31] Reid D B. An algorithm for tracking multiple targets. IEEE Transactions on Automatic Control, 1979.

[32] Fortmann T E, Bar-Shalom Y, Scheffe M. Sonar tracking of multiple targets using joint probabilistic data association. IEEE Journal of Oceanic Engineering, 1983.

[33] Zhang L, Li Y, Nevatia R. Global data association for multi-object tracking using network flows. CVPR, 2008.

[34] Kim C, Li F, Ciptadi A, Rehg J M. Multiple hypothesis tracking revisited. ICCV, 2015.

[35] Bergmann P, Meinhardt T, Leal-Taixé L. Tracking without bells and whistles. ICCV, 2019.

[36] Zhou X, Koltun V, Krähenbühl P. Tracking objects as points. ECCV, 2020.

[37] Wang Z, Zheng L, Liu Y, Li Y, Wang S. Towards real-time multi-object tracking. ECCV, 2020.

[38] Zeng F, Dong B, Zhang Y, Wang T, Zhang X, Wei Y. MOTR: End-to-end multiple-object tracking with transformer. ECCV, 2022.

[39] Sun P, Cao J, Jiang Y, et al. TransTrack: Multiple object tracking with transformer. arXiv:2012.15460, 2020.

[40] Maggiolino G, Ahmad A, Cao J, Kitani K. Deep OC-SORT: Multi-pedestrian tracking by adaptive re-identification. IEEE ICIP, 2023.

[41] Yang M, Han G, Yan B, et al. Hybrid-SORT: Weak cues matter for online multi-object tracking. AAAI, 2024.

[42] Sun P, Cao J, Jiang Y, et al. DanceTrack: Multi-object tracking in uniform appearance and diverse motion. CVPR, 2022.

[43] Zhou K, Yang Y, Cavallaro A, Xiang T. Omni-scale feature learning for person re-identification. ICCV, 2019.

[44] Clerc M, Kennedy J. The particle swarm — explosion, stability, and convergence in a multidimensional complex space. IEEE Transactions on Evolutionary Computation, 2002.

[45] Trelea I C. The particle swarm optimization algorithm: convergence analysis and parameter selection. Information Processing Letters, 2003.

[46] Ratnaweera A, Halgamuge S K, Watson H C. Self-organizing hierarchical particle swarm optimizer with time-varying acceleration coefficients. IEEE Transactions on Evolutionary Computation, 2004.

[47] Liang J J, Qin A K, Suganthan P N, Baskar S. Comprehensive learning particle swarm optimizer for global optimization of multimodal functions. IEEE Transactions on Evolutionary Computation, 2006.

[48] Sun J, Feng B, Xu W. Particle swarm optimization with particles having quantum behavior. IEEE Congress on Evolutionary Computation, 2004.

[49] Dorigo M, Maniezzo V, Colorni A. Ant system: optimization by a colony of cooperating agents. IEEE Transactions on Systems, Man, and Cybernetics, Part B, 1996.

[50] Karaboga D, Basturk B. A powerful and efficient algorithm for numerical function optimization: artificial bee colony (ABC) algorithm. Journal of Global Optimization, 2007.

[51] Mirjalili S, Lewis A. The whale optimization algorithm. Advances in Engineering Software, 2016.

[52] Mirjalili S, Gandomi A H, Mirjalili S Z, Saremi S, Faris H, Mirjalili S M. Salp swarm algorithm: A bio-inspired optimizer for engineering design problems. Advances in Engineering Software, 2017.

[53] Hansen N, Ostermeier A. Completely derandomized self-adaptation in evolution strategies. Evolutionary Computation, 2001.

[54] Jones D R, Schonlau M, Welch W J. Efficient global optimization of expensive black-box functions. Journal of Global Optimization, 1998.

[55] Snoek J, Larochelle H, Adams R P. Practical Bayesian optimization of machine learning algorithms. NeurIPS, 2012.

[56] Hutter F, Hoos H H, Leyton-Brown K. Sequential model-based optimization for general algorithm configuration. LION, 2011.

[57] Jamieson K, Talwalkar A. Non-stochastic best arm identification and hyperparameter optimization. AISTATS, 2016.

[58] Li L, Jamieson K, DeSalvo G, Rostamizadeh A, Talwalkar A. Hyperband: A novel bandit-based approach to hyperparameter optimization. JMLR, 2018.

[59] Falkner S, Klein A, Hutter F. BOHB: Robust and efficient hyperparameter optimization at scale. ICML, 2018.

[60] Jin Y. Surrogate-assisted evolutionary computation: Recent advances and future challenges. Swarm and Evolutionary Computation, 2011.

[61] Deb K, Jain H. An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, Part I: Solving problems with box constraints. IEEE Transactions on Evolutionary Computation, 2014.

[62] Zhang Q, Li H. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. IEEE Transactions on Evolutionary Computation, 2007.

[63] Breiman L. Random forests. Machine Learning, 2001.

[64] Rasmussen C E, Williams C K I. Gaussian Processes for Machine Learning. MIT Press, 2006.

[65] Demšar J. Statistical comparisons of classifiers over multiple data sets. JMLR, 2006.

[66] Branke J, Deb K, Dierolf H, Osswald M. Finding knees in multi-objective optimization. Parallel Problem Solving from Nature (PPSN VIII), 2004.

[67] Zitzler E, Thiele L. Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach. IEEE Transactions on Evolutionary Computation, 1999.

[68] Alatas B, Akin E, Ozer A B. Chaos embedded particle swarm optimization algorithms. Chaos, Solitons and Fractals, 2009.

Acknowledgements

[Write your acknowledgements here.]